Masaryk University
Faculty of Informatics



Performance analysis of Sorting Algorithms

Bachelor's Thesis

Matej Hulín

Brno, Fall 2017

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Matej Hulín

Advisor: prof. RNDr. Ivana Černá, CSc.

Acknowledgement

I would like to thank my advisor, prof. RNDr. Ivana Černá, CSc. for her advice and support throughout the thesis work.

Abstract

The aim of the bachelor thesis was to provide an extensive overview of sorting algorithms ranging from the pessimal and quadratic time solutions, up to those, that are currently being used in contemporary standard libraries. All of the chosen algorithms and some of their variants are described in separate sections, that include pseudocodes, images and discussion of their asymptotic space and time complexity in the best, average and the worst case.

The second and main goal of the thesis was to perform a performance analysis. For this reason, the selected algorithm variants, that perform optimally, have been implemented in C++. All of them have undergone several performance analysis tests, that involved 8 sequence types with different kind and level of presortedness.

The measurements showed, that timsort is not necessarily faster than mergesort on sequences, that feature only very limited amount of presortedness. In addition, the results revealed, that timsort is competetive with quicksorts on sequences of larger objects, that require a lot of comparisons. Secondly, we have found a single case, where the heapsort wasn't the worst performer. On sequences, that have a constant number of unique elements, specifically strings of length 32, it was able to outperform mergesort. Lastly, we have measured modified algorithms, that used binary insertion sort on constant-sized subsequences. From among all tested algorithms, the major running time differences could be observed for mergesort.

Keywords

Performance analysis, Sorting algorithms, Quicksort, Heapsort, Bottom-up heapsort, Mergesort, Bottom-up mergesort, Top-down mergesort, Insertion sort, Binary Insertion sort, Timsort, Stoogesort, Bogosort, Time complexity, Space complexity

Contents

Introduction **1**

1 Comparison-based sorting **3**
1.1 *Insertion sort* . . . 3
1.1.1 Description . . . 3
1.1.2 Asymptotic time and space analysis . . . 4
1.2 *Stooge sort* . . . 7
1.2.1 Description . . . 7
1.2.2 Correctness argument . . . 7
1.2.3 Asymptotic time and space analysis . . . 8
1.3 *Heapsort* . . . 11
1.3.1 Heap structure . . . 11
1.3.2 Top-down heap building . . . 12
1.3.3 Bottom-up heap building . . . 14
1.3.4 Bottom-up heapsort . . . 15
1.3.5 Asymptotic time and space analysis . . . 17
1.4 *Quicksort* . . . 19
1.4.1 Description . . . 19
1.4.2 Hoare's partitioning scheme . . . 20
1.4.3 Last pivot . . . 20
1.4.4 Random pivot . . . 21
1.4.5 Median of three pivot . . . 21
1.4.6 Asymptotic time and space complexity . . . 22
1.5 *Bogosort* . . . 26
1.5.1 Description . . . 26
1.5.2 Asymptotic time and space analysis . . . 27
1.6 *Merge sort* . . . 28
1.6.1 Introduction . . . 28
1.6.2 Merging . . . 28
1.6.3 Top-down mergesort . . . 29
1.6.4 Bottom-up mergesort . . . 30
1.6.5 Asymptotic time and space analysis . . . 31
1.7 *Timsort* . . . 33
1.7.1 Adaptive sorting . . . 33
1.7.2 Runs . . . 33

1.7.3 Merge Pattern 35
1.7.4 Merging runs 36
1.7.5 Galloping mode 37
1.7.6 Pseudocodes 37
1.7.7 Asymptotic time and space analysis 41

2 Performance analysis **45**
2.1 *Implementation* 45
2.2 *Testing environment* 46
2.3 *Data generation* 47
2.4 *Interface* 49
2.5 *Data evaluation* 50
2.5.1 The first part of measurings 51
2.5.2 The second part of measurings 51
2.5.3 The third part of measurings 63
2.5.4 Improvements using binary insertion sort 76
2.5.5 Conclusion points and final thoughts 77

3 Conclusion **83**

Bibliography **85**

Appendices **89**

Introduction

The area of sorting algorithms is an integral part of the theoretical computer science, that has been present since the early days of the Information age. During its evolution fostered by many researchers around the globe, it gave birth to many brilliant ideas solving this fundamental computational problem. The main tugger, namely competetivness, who will develop the fastest algorithm made this progress possible. Some of their findings and engineering mastery found its place in practice while others brought theoretical value and new ways of thinking to the fore.

One of the goals of the bachelor thesis was to provide an overview of comparison-based sorting algorithms, that range from the pessimal [1] ones, up to those, that are currently being used in standard libraries. In the first chapter, we have concentrated on sorting algorithms, some of which hold on tightly to $\Omega(n \log n)$ information-theoretic bound:

- Insertion sort
- Heapsort
- Quicksort
- Stoogesort
- Bogosort
- Mergesort
- Timsort

All of the listed algorithms and their variants were described in separate sections, that also included pseudocodes and discussion of their distinctive features. Among them particularly:

- Time complexity in the worst, average and best case
- Space complexity
- Stability – A sorting algorithm is stable, if on all randomly permuted sequences K applies following: $\forall(i, j) \in \{1, \ldots, |K|\}^2$, such that $i < j$ and $K[i]$ is equal to $K[j]$, the sorting algorithm

creates each time a non-decreasing permutation π, where all pairs (i, j) from K have some pair of indices (k, l) in π, such that $k < l$.

The last goal of this thesis was to examine how the sorting algorithms compete with each other. In the second chapter, all the presented sorting algorithms have undergone several performance tests, that involved 2 element types and 8 sequence types with different kind and level of presortedness. Some of these sequence types were even parametrized by additional variable, that depending on a sequence type even more particularized the level of presortedness.

Finally, all of the obtained measurements were closely examined, whether they are consistent with the proven asymptotic bounds and discussed with use of charts.

1 Comparison-based sorting

A comparison-based sorting, as it is self-evident, denotes a category of sorting algorithms, that use solely comparisons to gain order information about input. It requires, that all of the sorted elements are comparable.

The comparison-based sorting is one of the very well explored areas and the lower bound for required comparisons has been proved by decision trees to be $\Omega(n \log n)$ [2]. The following sections describe algorithms, based on comparisons. Many of them, like merge sort or heapsort are asymptotically optimal, others like quicksort manifest $\Omega(n \log n)$ bound in average case.

1.1 Insertion sort

1.1.1 Description

Insertion sort is a simple and relatively efficient comparison-based algorithm, even though it is not asymptotically optimal due to $\mathcal{O}(n^2)$ worst case bound. The following pseudocode presents the INSERTSORT function, that takes a sequence K of length n as a parameter:

Algorithm 1 Insertion sort

```
1: function INSERTSORT(K)
2:     for j ← 2 to |K| do
3:         key ← K[j]
4:         i ← j − 1
5:         while i > 0 ∧ K[i] > key do
6:             K[i + 1] ← K[i]
7:             i ← i − 1
8:         end while
9:         K[i + 1] ← key
10:    end for
11: end function
```

The Insertion sort (1) works as follows. It begins with an input sequence K, of which the first element k_1 trivially constitutes a sorted sequence of size 1. The outer loop then iterates over nonsorted elements

$[k_2, k_3, \ldots, k_n]$. In each iteration, the picked element k_j, which happens to be the first nonsorted element is iteratively compared to elements in the sorted sequence $[k_{\pi_1}, k_{\pi_2}, \ldots, k_{\pi_{j-1}}]$ in reverse order. The inner while cycle on lines 5–8 realizes such search, until either $i = 0$ or $K[i] \leqslant key$. The former case implies, that the key is the smallest key so far and line 9 sets $K[0] = key$. In the latter case, the key is placed directly after the last lower or equal element, preserving relative order of equal elements in input sequence. Consequently, the stability is achieved. In addition, all of the elements, for which $K[i] > key$ applies, are involved in element movement and each is pushed by one, making space for k_j.

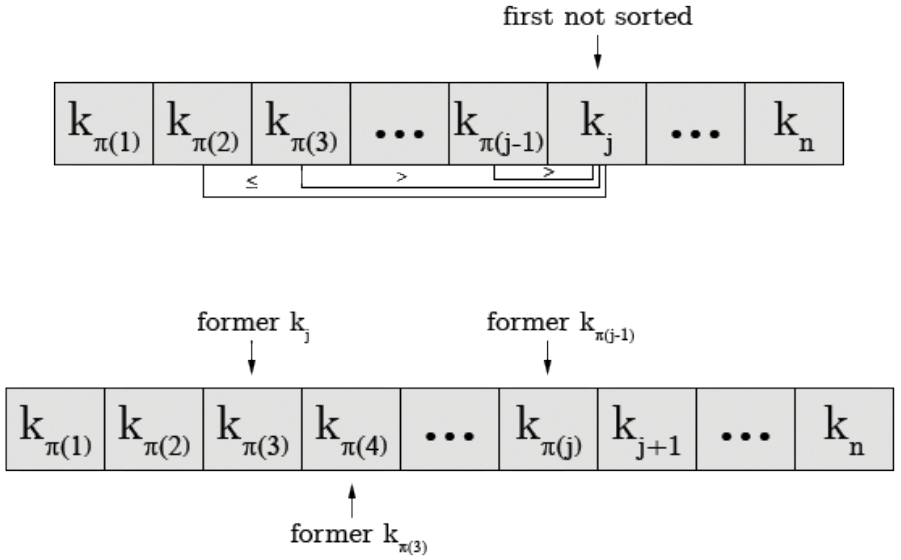


Figure 1.1: The state of sorting before and after one iteration of outer loop. The upper part pictures the process of searching a correct position for element k_j and failed condition of inner loop, after which the correct place is found. The lower part displays the element order after the iteration.

1.1.2 Asymptotic time and space analysis

The time complexity of insertion sort is constituted by a number of comparisons, element moves, and size n of input sequence. The number

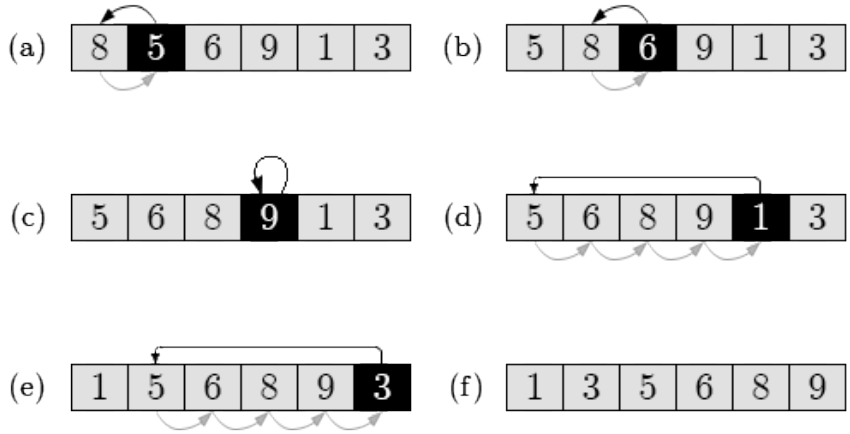


Figure 1.2: The process of sorting sequence $K = [8, 5, 6, 9, 1, 3]$. Each of **(a)**–**(f)** represent one iteration of outer loop, after which the sorted sequence is one larger. The value in a black rectangle holds a number, which is currently being inserted into sorted sequence. The arrows represent movements of corresponding elements.

of comparisons can be easily seen as a sum of arithmetic progression:

$$\begin{aligned} \sum_{i=1}^{n-1} i &= \frac{n(n-1)}{2} \\ &= \mathcal{O}(n^2) \end{aligned} \tag{1.1}$$

The same applies to a number of element moves. In each iteration of outer loop, at most a number of currently sorted elements has to be moved. Accordingly, the worst case asymptotic time is $\mathcal{O}(n^2)$. Analogously, we can find asymptotic bound by executing insertion sort as a recurrence, where in each recursive call, the length of sorted sequence is increased by one. The recursion is following:

$$T(n) \quad \begin{cases} 1 & \text{for } n = 1 \\ T(n-1) + n & \text{for } n > 1 \end{cases}$$

The derived time complexity constitutes a strict bound for the insertion sort running times. It also applies to the improved version, that uses binary search to search for a correct position of elements. Even though the number of comparisons is lower than quadratic, the costs for swaps remain the same.

The average case complexity can be also derived easily. It suffices to realize, that in each cycle of outer loop, there is in average $\frac{(i-1)}{2}$ elements, which are greater than k_i. Hence, it doesn't improve the worst case time 1.1 at all.

On the other hand, it performs quite well on nearly sorted sequences. It can be seen, that for already sorted sequences suffice just n steps. In each iteration of the outer loop, the inner while loop terminates immediately and altogether linear number of comparisons and element movements in input size are performed. Thus, the best case time is $\mathcal{O}(n)$.

In terms of space complexity, the insertion sort requires just constant number of additional memory cells. Hence, it has $\mathcal{O}(1)$ space in worst case.

1.2 Stooge sort

1.2.1 Description

Another example of comparison-based sorting algorithm is stooge sort [3]. As its name suggests, it provides rather inefficient way of sorting, which is inferior even to insertion sort. It achieves asymptotically non-optimal worst case bound $\mathcal{O}(n^{\log 3/\log 1.5}) = \mathcal{O}(n^{2.7095})$. Calling Stoogesort($K$, 1, $|K|$), on randomly permuted sequence K, with indices i and j denoting the first and last element, the following steps are executed:

Algorithm 2 Stooge sort

```
1: function Stoogesort(K, i, j)
2:     if K[i] > K[j] then
3:         K[i] ↔ K[j]
4:     end if
5:     if i + 1 ⩾ j then
6:         return
7:     end if
8:     k ← ⌊(j − i + 1)/3⌋
9:     Stoogesort(K, i, j − k)
10:    Stoogesort(K, i + k, j)
11:    Stoogesort(K, i, j − k)
12: end function
```

- If the first value is larger than the last one, swap them
- If there are at least 3 elements:
  - Stoogesort the initial $\lceil 2/3 \rceil$ values
  - Stoogesort the last $\lceil 2/3 \rceil$ values
  - Stoogesort the initial $\lceil 2/3 \rceil$ values

1.2.2 Correctness argument

The stooge sort pseudocode looks simple, but the correctness may not be that apparent. We will prove it by induction. The correctness of base cases with input size of one or two is trivial. Now, suppose, that the

presented stooge sort can sort sequences smaller than $|K[i,\ldots,j]|$. In induction step, we will prove that it can also sort correctly $K[i,\ldots,j]$.

After executing stooge sort(line 8) on first $\lceil 2/3 \rceil$ elements, the sequence $K[i,\ldots,j-k]$ is sorted (by assumption). Important observation here is, that:

$$\forall(l,t).(l \in \{i,\ldots,i+k\} \wedge t \in \{i+k+1,\ldots,j-k\})\ K[l] \leqslant K[t]$$

In the sequel, we will denote such property by:

$$K[i,\ldots,i+k] \leqslant K[i+k+1,\ldots,j-k]$$

After executing line 9, the k greatest elements within initial $2k$ elements are sorted with upper k elements. The sequence $K[i+k,\ldots,j]$ is then sorted and evidently:

$$K[i+k,\ldots,j-k] \leqslant K[j-k+1,\ldots,j]$$

As a consequence, by combining afore-mentioned, we can conclude, that

$$K[i,\ldots,i+k] \leqslant K[j-k+1,\ldots,j]$$

$$K[i,\ldots,j-k] \leqslant K[j-k+1,\ldots,j]$$

the last k elements are already in place. However, after executing stooge sort on the last $\lceil 2/3 \rceil$ elements, we can't assume any order of elements in $K[i,\ldots,j-k]$, besides that $K[i,\ldots,i+k]$ and $K[i+k+1,\ldots,j-k]$ constitute separately sorted sequences. Thus, another round of stoogesort for first $\lceil 2/3 \rceil$ elements is required. Finally, after executing line 11, $K[i,\ldots,j-k]$ is sorted and by putting together our observations, we can see that

$$K[i,\ldots,i+k] \leqslant K[i+k+1,\ldots,j-k] \leqslant K[j-k+1,\ldots,j]$$

is a completely sorted sequence $K[i,\ldots,j]$.

1.2.3 Asymptotic time and space analysis

Asymptotic time complexity analysis is straightforward. The algorithm first does constant time computation on lines 2–7 and then recursively calls itself three times on input of size $\lceil 2/3 \rceil$. From a number of recursive

The cost of the entire recursion tree: $3^{\log_{3/2} n} + \sum_{i=0}^{\log_{3/2}(n)-1} 3^i = n^{\log_{3/2} 3} + \frac{1}{2}\left(3^{\log_{3/2} n} - 1\right) = \mathcal{O}(n^{\log_{3/2} 3})$

Figure 1.3: Stooge sort's recursion tree

calls, comparisons and element moves in each call, we can state its recurrence $T(n)$:

$$T(n) = \begin{cases} \Theta(1), & \text{for } n \in \{1, 2\} \\ 3T(2n/3) + \Theta(1) & \text{for } n > 2 \end{cases}$$

One of the ways to obtain time complexity is to put in use master theorem. Assessing the coefficients in the recurrence, we can see that the total cost is dominated by the cost in leaves and that the first case of master theorem applies. The constant divide and combine cost of $T(n)$, $f(n) = 1$ is clearly polynomially smaller than $n^{\log_{3/2} 3}$, thus the

$$\begin{aligned} f(n) &= \mathcal{O}(n^{\log_{3/2} 3-\epsilon}) \text{ for some } \epsilon > 0 \\ &= \mathcal{O}(n^{\frac{\log 3}{\log 1.5}-\epsilon}) \\ &= \mathcal{O}(n^{2.7095-\epsilon}) \text{ for } \epsilon \approx 2.7095 \end{aligned}$$

worst case running time of stooge sort is $T(n) = \Theta(n^{2.7095})$.

This bound applies also to the best case running time, because the algorithm descends down to all leaves in the recursion tree independently on values and order of elements in sequence.

Due to the identical worst and best case complexities, we can conclude, that the average case complexity won't be any different.

The stooge sort, as presented, doesn't use any extra memory, except for the logarithmic space required by recursion stack.

The stability is not preserved during sorting. The argument, that supports such claim is the sequence $[9, 2, 2, 9, 1]$.

1.3 Heapsort

Another competitor among comparison-based sorting algorithms is a heapsort. It was first invented by J. W. J. Williams in 1964 [4] and the same year improved by R. W. Floyd [5]. Since then, many research papers introducing other improvements were published [6, 7]. The main purpose of these were to lower a number of comparisons, so that heapsort could theoretically compete with quicksort.

1.3.1 Heap structure

In the centre of heapsort lies a heap structure. The heap can be described as a complete binary tree, whose nodes map to unique array indices and represent their values. If the given array is non-empty, the first element denotes a root of the heap. The left-child, right-child and parent node indices can be computed as:

Algorithm 3 Left-child

```
1: function LEFT-CHILD(i)
2:     return 2i
3: end function
```

Algorithm 4 Right-child

```
1: function RIGHT-CHILD(i)
2:     return 2i + 1
3: end function
```

Algorithm 5 Parent

```
1: function PARENT(i)
2:     return ⌊i/2⌋
3: end function
```

In general, there are two kinds of heaps, ***min-heap*** and ***max-heap***. Both of them must satisfy a heap property invariant, in order to be a correct heap structure. The kind of conditions, which apply, ultimately

depends on the kind of heap. Thus, for all nodes of min-heap other than root:

$$K[\textsc{Parent}(i)] \leqslant K[i]$$

and similarly for max-heap:

$$K[\textsc{Parent}(i)] \geqslant K[i]$$

From the provided invariant, it is clear, that the root node must contain the smallest/greatest element for min-heap/max-heap, respectively. In the heapsort's history, two approaches to heap building were created, namely **top-down** and **bottom-up**. In the following subsections, both of them will be briefly presented and compared in terms of efficiency.

The process of sorting the input sequence K can be divided into *construction* and *selection* phase. First of all, the heap structure A is constructed on top of K, so that root contains the smallest/greatest element and both child heaps satisfy the min/max heap property. In the *selection phase* the following steps repeated. First, the root value is swapped with the last heap element ($A[A.heapsize]$). Subsequently, $A.heapsize$ is lowered by one and a *reheap* function is called on root. The *reheap* function provided, that the left and right subtrees form a proper heaps, places a root value into correct position and reestablishes the heap invariant. To understand how the heap A is constructed and how elements are sorted, the following functions will be presented:

- Build-Max-Heap(K) – Creates heap A on top of K, satisfying the heap property in $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ using bottom-down or top-down approach respectively.
- Reheap – Restores the invariant for the heap rooted at supplied node
- Heapsort(K) – Sorts K using last two functions in $\mathcal{O}(n \log n)$ worst case time.

1.3.2 Top-down heap building

The top-down approach rests in its reheap function, which is also called *sift-up* function. Given some start index and root index, *sift-up* bubbles up and restores the heap invariant. Then, during the heap *construction*

phase, the heap-building function passes through heap levels in top-down manner, each from left to right calling *sift-up* function for each node. This way, beginning with a trivial heap comprised of lone root, we can consecutively insert elements. Unfortunately, this approach doesn't

Algorithm 6 Build-Max-Heap(K)

```
1: function BUILD-MAX-HEAP(K)
2:     A.heapsize ← K.length
3:     for j ← 2 to A.heapsize do
4:         SIFT-UP(A, j, 1)
5:     end for
6: end function
```

Algorithm 7 Sift-up

```
1: function SIFT-UP(A, i, root)
2:     while i > root do
3:         parent ← PARENT(i)
4:         if A[parent] < A[i] then
5:             A[parent] ↔ A[i]
6:             i ← parent
7:         else
8:             return
9:         end if
10:    end while
11: end function
```

boast with the optimal time complexity and achieves only $\mathcal{O}(n \log n)$ time. To substantiate such claim, it is important to note, that with the growing heap depth, the number of $sift-up$ calls increases exponentially. It means, that the most numerous node-level makes at least $\lfloor n/2 \rfloor$ *sift-up* calls, each with $\mathcal{O}(\log n)$ complexity. As a result, the upper bound can be determined to be $\mathcal{O}(n \log n)$. The following summation confirms such bound by counting a number of comparisons:

$$\begin{aligned}\sum_{l=0}^{\lfloor \log n \rfloor - 1} l2^l &= 2^{\lfloor \log n \rfloor}\lfloor \log n \rfloor - 2(2^{\lfloor \log n \rfloor} - 1) \\ &= n\lfloor \log n \rfloor - 2n + 2\end{aligned} \tag{1.2}$$

The number of interchanges incurs a similar costs, so the upper bound is not influenced at all. In case of an already built heap, a linear number of *sift-up* calls, each with a constant number of comparisons contribute to the best case linear cost.

By making no assumptions about integer distributions, we can say, that each path of length l from some *start* node to *root* contains on average $l/2$ values larger than one in $A[start]$. Either way, by summing comparisons in this average case, we get the asymptotic bound equal to the worst case one.

The top-down heap building operates exclusively on the input sequence K and for this reason, it is in-place.

1.3.3 Bottom-up heap building

The bottom-up heap building improves upon top-down one and achieves $\mathcal{O}(n)$ worst case time complexity by using bottom-up approach, when performing the reheap. The first such representative was introduced in [5], which is probably the most well-known version. However, in this section, heap building will use bottom-up reheap function as presented in [7]. Next, the *modus operandi*[1] of the bottom-up reheap function shall be clarified.

Algorithm 8 Build-max-heap

```
1: function BUILD-MAX-HEAP(K)
2:     A.heapsize ← K.length
3:     for j = ⌊A.heapsize/2⌋ downto 1 do
4:         BOTTOM-UP-REHEAP(A, i)
5:     end for
6: end function
```

The Bottom-up-reheap works as follows:

1. method or mode of operation

Algorithm 9 Bottom-up-reheap

```
1: function BOTTOM-UP-REHEAP(A, i)
2:     leaf ←LEAF-SEARCH(A, i)
3:     position ←BOTTOM-UP-SEARCH(A, i, leaf)
4:     INTERCHANGE(A, i, position)
5: end function
```

1. First, the path of maximum sons, which is also referred to as special path, is found. This special path is defined in [8] as:*"A path of maximum sons of a heap is the largest son of the root concatenated with the path of maximum sons of the subheap with that element as the root"*. Therefore, the LEAF-SEARCH proceeds downward from $A[i]$ through the tree structure, at each level makes one comparison between root's children and takes a path of the greater one. This doesn't involve any element swaps. The Leaf-Search function returns a leaf node index.

2. In the second stage, the BOTTOM-UP-SEARCH function climbs up the heap similarly to the *sift-up* function and searches a correct *position* for $A[i]$ in a special path beginning at *leaf*.

3. Lastly, the INTERCHANGE moves values in sequence $A[position]$ up to some child of $A[i]$ on special path upward by one and inserts former $A[i]$ into $A[position]$.

The worst case time complexity of the algorithm 8 can be analogously derived as for *sift-up* (1.3.2). It is sufficient to observe, that as a number of nodes increases exponentially with the growing heap depth, the cost of BOTTOM-UP-REHEAP lowers, too. Thus, the deepest level with $\lfloor n/2 \rfloor$ nodes makes a linear number of calls, each costing just constant. Summing over each node's subtree depth gives the number of comparisons:

$$\sum_{l=0}^{\log n} \left\lceil \frac{n}{2^{l+1}} \right\rceil l = \mathcal{O}(n)$$

1.3.4 Bottom-up heapsort

In contemporary, there are many variations of heapsort, that differ in kind of reheap function used in the *construction* and *selection* phase.

Algorithm 10 Leaf-search

```
1: function LEAF-SEARCH(A, i)
2:     j ← LEFT-CHILD(i)
3:     while j < A.heapsize do
4:         if A[j + 1] ⩽ A[j] then
5:             j ← j + 1
6:         end if
7:         j ← LEFT-CHILD(j)
8:     end while
9:     if j > A.heapsize then
10:        j ← PARENT(j)
11:    end if
12:    return j
13: end function
```

Algorithm 11 Bottom-up-search

```
1: function BOTTOM-UP-SEARCH(A, i, j)
2:     while (i < j) ∧ (A[i] ⩽ A[j]) do
3:         j ← PARENT(j)
4:     end while
5:     return j
6: end function
```

Algorithm 12 Interchange

Require: $j \geqslant i$

```
1: function INTERCHANGE(A, i, j)
2:     l ← ⌊log (j/i)⌋
3:     tmp ← A[i]
4:     for k ← l − 1 to 0 do
5:         A[j/2^{k+1}] ← A[j/2^k]
6:     end for
7:     A[j] ← tmp
8: end function
```

The bottom-up heapsort, that we will approach has both phases based on bottom-up reheap function. As a first step, it creates a heap structure on top of K in linear time. Then, in the *selection* phase, it repeatedly swaps the root of the heap $A[1]$ with the last element $A[A.heapsize]$, lowers $A.heapsize$ and performs reheap on the root. At the end of each iteration, the sorted sequence at the back of K grows by one. Finally, the loop terminates on the $A.heapsize$ equal to 1 and the whole K is sorted.

The bottom-up reheaping is an efficient approach, which compared to other solutions uses the fact, that the swapped leaves are very likely to sink back down. Basically, it allows to lower a number of element moves and comparisons, by first searching a special path and only then climbing up and placing the leaf.

Algorithm 13 Bottom-up heapsort

```
1: function BOTTOM-UP-HEAPSORT(K)
2:     BUILD-MAX-HEAP(K)
3:     while A.heapsize > 1 do
4:         A[1] ↔ A[A.heapsize]
5:         A.heapsize ← A.heapsize − 1
6:         BOTTOM-UP-REHEAP(A, 1)
7:     end while
8: end function
```

1.3.5 Asymptotic time and space analysis

The worst case asymptotic bound is for the most part formed by the *selection phase*. In each of n iterations, the BOTTOM-UP-REHEAP with $\mathcal{O}(\log n)$ complexity is called exactly once. Therefore, the worst case time is $\mathcal{O}(n \log n)$. The *construction phase* contributes to this bound only negligible amount. Even more precise bound was shown by Wegener. By his estimates, the bottom-up heapsort requires $1.5n \log n + \mathcal{O}(n)$ [7, p. 16] comparisons in the worst case and just $n \log n + \mathcal{O}(n)$ [7, p. 15] on the average.

Regarding space complexity, the heapsort operates on top of underlying array and requires just constant amount of additional space $\mathcal{O}(1)$.

Heapsort is not a stable sorting algorithm. Both *construction* and *selection* phases don't preserve the relative ordering of equal elements. Even when an input instance is already sorted, the order of equal elements is reversed in the *selection* phase.

1.4 Quicksort

Quicksort is a comparison-based divide and conquer sorting algorithm, that was first time published in 1961 by Hoare [9]. Due to its prominence, it remained a vastly used algorithm, that performs exceptionally on all kinds of inputs.

1.4.1 Description

Similarly to mergesort, it is based on sequence partitioning and assumption, that by sorting smaller segments, the whole input can be sorted. However, the quicksort's approach to partitioning is quite different. The first step of partitioning is based on chosen strategy to choose an element from among multitude of keys contained in input. In literature, such element is frequently referred to as *bound* or *pivot*. Its aim is then to rearrange sequence, so that all elements before the pivot are lower or equal and all elements after are greater or equal. Seemingly, in the partitioned sequence, the pivot is in correct position, which is given by its rank and possible deviation caused by a number of equal elements. To complete the sorting, the quicksort has to be called recursively on two subsequences, that are separated by pivot and which are strictly smaller than the length of input in a previous call. This way, by calling quicksort on smaller instances, it will eventually hit problems of trivial size and start backtracking on recursion tree. However, the recursive algorithm may sometimes lead to stack overflow and become rather dangerous part of software, when used unwisely. To prevent such undesirable situation, quicksort can be implemented iteratively. This section provides pseudocodes for iterative quicksorts, that simulate recursion using explicitely created stack. In 14, 15 and 16, each iteration of outer while loop corresponds to one recursive call, albeit, pushing indices to stack replaces recursive calls.

The cost of sorting outlined in the previous paragraph, is highly dependent on two factors – the choice of suitable pivot and the way partitioning is carried out. A choice of suitable pivot lowers the depth of recursion and efficient partitioning lowers costs per recursive step. In particular, the partitioning scheme proposed by Hoare [10] is optimal and widely used.

1.4.2 Hoare's partitioning scheme

The Hoare's partitioning was for the first time introduced in "Quicksort" [10], along with a detailed cost analysis. It provides efficient partitioning using technique, which is called *crossing pointers*. The partitioning works as follows. First, to prevent pivot from comparing to itself, it is swapped with element at the lowest or the highest index. Subsequently, we initialize two indices i and j depending on the position of pivot. If the pivot is positioned at the lowest index, i is initialized with the lowest index and j with one past the highest index. Otherwise, i is initialized with one lower than the lowest index and j with the highest index. Then, in the infinite loop, we iterate over K and compare $K[{+}{+}i]$ to the chosen pivot. Each time $K[{+}{+}i]$ is lower, i is stepped up, until it is equal to the highest index or some element greater or equal to pivot is encountered. A similar search is also executed from the back of K using j, until some element lower or equal is found. Then, each iteration is concluded by a swap of $K[i]$ and $K[j]$. Evidently, these two elements don't have to even invalidate invariant to get swapped (equality to pivot). This is due to Sedgewick [11, p. 292], who showed, that it helps to avoid quadratic complexity, when input contains just a constant fraction of distinct elements. On the contrary, if indices cross each other, the swap is omitted and infinite loop is left. Overall, there are two cases, how pointers, in our case indices i and j can cross each other. Either $i = j$, or $j + 1 = i$. In both cases, all elements wih index lower or equal to j have values, that are smaller or equal to pivot. To conclude the partitioning process, the pivot has to be swapped to position, that approximately corresponds to its rank. This can be accomplished by swapping pivot positioned at the highest index with $K[i]$ or swapping pivot positioned at the lowest index with $K[j]$.

To partition the input K, at most linear number of comparisons and swaps are required. Hence, the Hoare's partitioning has the worst case time complexity $\mathcal{O}(n)$.

1.4.3 Last pivot

In contemporary, there are plenty of strategies, that choose a pivot with varying degree of suitability. This crucial choice may considerably optimize the proportionality of partitioned subsequences and in turn

lower a depth of recursion. One of the strategies, that is simple and also trivial to implement, is based on repeatedly choosing the last element. In most of the cases, this strategy performs fine, although there are some strings attached to it. Each time a last element is chosen as pivot, there's only $1/n$ chance of choosing the greatest element, that results in a very unproportional partitions. This is almost favourable, however, for already sorted sequences, this pivot strategy leads to the linear recursion depth, where in each step, a linear cost is incurred by partitioning. Thus, it immediately slides to the worst case of $\mathcal{O}(n^2)$.

On the other hand, the average case achieves the complexity $\mathcal{O}(n \log n)$. This is caused predominantly by the mean rank of the selected pivot, that is equal to $\frac{n+1}{2}$. In other words, most of the time, the partitions are nearly proportional and recursion depth is just logarithmic. Combined with the partitioning costs, the average bound is obtained.

1.4.4 Random pivot

Another approach, that completely mitigates the case of sorted (or reverse-sorted) input is accomplished by choosing a random pivot from the range $[lo, \ldots, hi]$. In average case, it carries out $1.386n \log n$ comparisons [10, p. 12], that also constitute the upper bound for the number of swaps. Hence, the average case time complexity is $\mathcal{O}(n \log n)$.

1.4.5 Median of three pivot

In the early days of the quicksort, Hoare proposed another strategy, that involves using median as a pivot. Due to its impractical computational complexity, Singleton [12] suggested a good approximation, that computes median from among three elements, specifically the first, middle and the last one. This strategy excels especially in cases, when the input is sorted or reverse-sorted. By picking the middle element, it ensures that partitioning will use median and create a proportional partitions.

The quicksort, that partitions with median of three strategy, performs just $1.188n \log n$ comparisons on average [13, 12].

1.4.6 Asymptotic time and space complexity

The quicksort's worst case bound is obtained by a series of bad pivot choices, resulting in unproportional partitioning. Thus, in each recursive step, there are two calls to Quicksort – one that has a constant input size and one, that has input size lowered by the constant. As a consequence, there is a linear number of steps with linear complexity, that contribute to $\mathcal{O}(n^2)$.

The average case complexity using any of the described pivot strategies leans towards $\mathcal{O}(n \log n)$, although with different constant factors.

The space complexity of the quicksort never exceeds $\mathcal{O}(\log n)$ thanks to a simple trick, that involves calling itself on the smaller subsequence prior to the larger one. Then, whenever the the first call finishes, the stack space can be reclaimed, before proceeding to the second call [14, p. 849]. In iterative version, this corresponds to pushing the larger subsequence before the smaller one.

The quicksort is not stable. The relative ordering is not preserved in partitioning subroutine, that swaps also equal elements.

Algorithm 14 Quicksort with last element pivot

```
 1: function QUICKSORT-LAST(K, lo, hi)
 2:     if hi − lo < 1 then
 3:         return
 4:     end if
 5:     stack π ← ∅
 6:     PUSH(π, (lo, hi))
 7:     while |π| > 0 do
 8:         (slo, shi)←POP(π)
 9:         if shi − slo < 1 then
10:             continue
11:         end if
12:         i ← slo − 1
13:         j ← shi
14:         pivot ← shi                          ▷ Choose last element as pivot
15:         while true do                                  ▷ Hoare partitioning
16:             while K[pivot] > K[++i] do
17:             end while
18:             while j > slo ∧ K[--j] > K[pivot] do
19:             end while
20:             if i ⩾ j then
21:                 break
22:             end if
23:             K[i] ↔ K[j]
24:         end while
25:         K[pivot] ↔ K[i]
26:         if shi − i > i − slo then              ▷ Pushing to the stack π
27:             PUSH(π, i + 1, shi)
28:             PUSH(π, slo, i − 1)
29:         else
30:             PUSH(π, slo, i − 1)
31:             PUSH(π, i + 1, shi)
32:         end if
33:     end while
34: end function
```

Algorithm 15 Quicksort with random pivot

```
 1: function QUICKSORT-RANDOM(K, lo, hi)
 2:     if hi − lo < 1 then
 3:         return
 4:     end if
 5:     stack π ← ∅
 6:     PUSH(π, (lo, hi))
 7:     while |π| > 0 do
 8:         (slo, shi)←POP(π)
 9:         if shi − slo < 1 then
10:             continue
11:         end if
12:         K[slo] ↔ K[RAND(slo, shi)] ▷ Choose random pivot and swap it
13:         pivot ← slo
14:         i ← slo
15:         j ← shi + 1
16:         while true do                                  ▷ Hoare Partitioning
17:             while i < shi ∧ K[pivot] > K[++i] do
18:             end while
19:             while K[--j] > K[pivot] do
20:             end while
21:             if i ⩾ j then
22:                 break
23:             end if
24:             K[i] ↔ K[j]
25:         end while
26:         K[pivot] ↔ K[j]
27:         if shi − j > j − slo then                      ▷ Pushing to the stack π
28:             PUSH(π, j + 1, shi)
29:             PUSH(π, slo, j − 1)
30:         else
31:             PUSH(π, slo, j − 1)
32:             PUSH(π, j + 1, shi)
33:         end if
34:     end while
35: end function
```

Algorithm 16 Quicksort with median of three pivot

```
 1: function QUICKSORT-MEDIAN(K, lo, hi)
 2:     if hi − lo < 1 then
 3:         return
 4:     end if
 5:     stack π ← ∅
 6:     PUSH(π, (lo, hi))
 7:     while |π| > 0 do
 8:         (slo, shi)←POP(π)
 9:         if shi − slo < 1 then
10:             continue
11:         end if
12:         mid ← slo + (shi − slo + 1)/2                ▷ Compute median pivot
13:         if K[slo] > K[mid] then
14:             K[slo] ↔ K[mid]
15:         end if
16:         if K[mid] > K[shi] then
17:             K[mid] ↔ K[shi]
18:             if K[slo] > K[mid] then
19:                 K[slo] ↔ K[mid]
20:             end if
21:         end if
22:         K[slo + 1] ↔ K[mid]
23:         i ← slo + 1
24:         j ← shi + 1
25:         pivot ← slo + 1
26:         while true do                                ▷ Hoare Partitioning
27:             while i < shi ∧ K[pivot] > K[++i] do
28:             end while
29:             while K[--j] > K[pivot] do
30:             end while
31:             if i ⩾ j then
32:                 break
33:             end if
34:             K[i] ↔ K[j]
35:         end while
36:         K[pivot] ↔ K[j]
37:         if shi − j > j − slo then                    ▷ Pushing to the stack π
38:             PUSH(π, j + 1, shi)
39:             PUSH(π, slo, j − 1)
40:         else
41:             PUSH(π, slo, j − 1)
42:             PUSH(π, j + 1, shi)
43:         end if
44:     end while
45: end function
```

1.5 Bogosort

1.5.1 Description

Bogosort, also called *stupid sort* or *permutation sort* is an awfully inefficient sorting algorithm and representative of pessimal algorithm design [1]. The algorithm adheres to the generate and test paradigm, which is a fundamental method of problem solving. It can be compared to repeated attempts of experiment, until a certain unique event eventually occurs. This is exactly how the bogosort behaves in its randomized version. In this section, we are going to restrict ourselves to this version, not the deterministic one.

Algorithm 17 Randomized Bogosort

```
1: function BOGOSORT(K)
2:     while !SORTED(K) do
3:         PERMUTE(K)
4:     end while
5: end function
```

As the pseudocode implies, the bogosort first tests whether the input sequence K is sorted. It takes at most $|K| - 1$ comparisons if sorted. Otherwise, the K is permuted uniformly at random, as presented in PERMUTE function. In each of $i \in \{1, \ldots, |K|\}$ iterations, i*th* element is swapped with element at position which is randomly picked from the range $[i, \ldots, |K|]$. The succession of permuting and testing continues until the sorted sequence is found.

Algorithm 18 Check if sequence K is sorted

```
1: function SORTED(K)
2:     for i ← 1 to |K| − 1 do
3:         if K[i] > K[i + 1] then
4:             return false
5:         end if
6:     end for
7:     return true
8: end function
```

The following permute algorithm can be also found in [15] and proved using loop invariant [16]. By being uniform, it guarantees, that each of

Algorithm 19 Permute K

```
1: function Permute(K)
2:     for i ← 1 to |K| do
3:         j ← Rand(i, |K|)
4:         K[i] ↔ K[j]
5:     end for
6: end function
```

$n!$ permutes appears with the same probability, what in turn greatly alleviates complexity analysis.

1.5.2 Asymptotic time and space analysis

Like other comparison-based algorithms, the bogosort depends on the number of comparisons and swaps. The best case time can be achieved, when K is already sorted. In this case, $n = |K|$ comparisons are carried out to check the sequence and leave the cycle. No other swaps are needed, hence the best case asymptotic time is $\mathcal{O}(n)$.

The worst case time complexity is more complex to derive, because bogosort is randomized algorithm. That means, that there's no guarantee, the algorithm will ever complete its computation. However, to prove, that expected running time is finite, it is sufficient to show that $\mathbb{E}[I] < \infty$ for I, which denotes a number of iterations until the input is sorted. Such proof was shown by Gruber; Holzer; Ruepp [17], who derived $\mathcal{O}((n-1)n!)$ worst case and $\Omega(n.n!)$ average case bounds.

The bogosort operates on top of input sequence and requires just constant additional space due to temporarily stored values in swap function.

The stability in this kind of pessimal algorithm was not much of a concern. In each iteration, the information about relative ordering of equal elements is lost during permuting and is not reflected in the resulting sequence.

1.6 Merge sort

1.6.1 Introduction

Mergesort is one of the oldest and time optimal comparison-based sorting algorithms. It was invented by J. von Neumann in 1945 [18, p. 42], but the first description and analysis comes from 1948's report of Goldstine; Von Neumann [19]. Since then, many variants have been studied. This section is based on top-down and bottom-up two-way mergesort (also called *straight two-way* mergesort [20, p. 162]) for internal sorting.

1.6.2 Merging

The corner-stone of the mergesort is its merging routine. Given two non-empty sorted subsequences, the merge function assembles two into sorted sequence in time, that is proportional to common size. Under assumption, that each sequence of length 1 is already sorted, it is possible to gradually create longer and longer sorted runs, until the whole input instance is sorted. The merging process works as follows. First, the subsequence $K[lo, \ldots, mid]$ is moved into temporary array J. Subsequently, the while loop iterates over J and if $J[i] \leqslant K[j]$, or all

Algorithm 20 Merge $K[lo, \ldots, mid]$ with $K[mid + 1, \ldots, hi]$

```
1: function MERGE(K, lo, mid, hi)
2:     J ← K[lo, ..., mid]
3:     i ← 1
4:     j ← mid + 1
5:     l ← lo
6:     while i < mid − lo + 1 do
7:         if (j > hi ∨ J[i] ⩽ K[j]) then
8:             K[l++] ← J[i++]
9:         else
10:            K[l++] ← K[j++]
11:        end if
12:    end while
13: end function
```

elements from $K[mid + 1, \ldots, hi]$ have been placed, it takes a branch,

that appends $J[i]$ to the sorted sequence, starting from the lowest index (lo). In the opposite case, if $j \leqslant hi$ and $J[i] > K[j]$, $K[j]$ is placed. After all, this preserves the stability under premise, that two consecutive subsequences are being merged. The case, when all of the elements in $J[1, \ldots, mid]$ are placed is trivial, because there is no element in J and $K[j]$ is the first element, that is greater or equal to the last one placed from J. Therefore, after the while cycle terminates, the $K[lo, \ldots, hi]$ constitutes a sorted sequence.

The space complexity of Merge is linear function proportional to the common size of runs to be merged. However, when sorting larger instances, it may become the serious setback. Therefore, a handful of in-place merge functions have been invented [21, 22, 23]. Unfortunately, the index manipulation introduces in practice non-trivial overhead, that makes the sorting process slower [21, p. 1]. As a consequence, the linear space complexity merge is preffered.

1.6.3 Top-down mergesort

Mergesort is the archetypal of *divide-and-conquer* design paradigm. In the *divide* phase, it divides input sequence into two equal-sized subsequences, until a trivial case of one element is reached in logarithmic number of steps. Then, in the *conquer* phase, the runs are subsequently merged into larger sequences in postorder fashion, terminating on the merge of initial two halves of K. In comparison to the bottom-up

Algorithm 21 Top-down Mergesort

```
1: function Top-down-mergesort(K, i, j)
2:     n ← (j − i + 1)
3:     if n > 1 then
4:         Top-Down-Mergesort(K, 1, ⌊n/2⌋)
5:         Top-Down-Mergesort(K, ⌊n/2⌋ + 1, n)
6:         merge(K, 1, ⌊n/2⌋, n)
7:     end if
8: end function
```

approach, the top-down features additional logarithmic space cost due to recursive calls. Either way, the space complexity remains linear.

1.6.4 Bottom-up mergesort

Another way of implementing mergesort is using iterative approach, which proceeds from the trivial subproblems (bottom) up, solving the larger ones. The bottom-up mergesort treats the input sequence K of length n as a list of n sublists, each trivially sorted. Then, in each of the logarithmic number of passes, it merges consecutive sorted subsequences in pairs (*two-way*). To show, that the described approach works, we can prove the correctness of 22 by the following loop invariant:

Algorithm 22 Bottom-up Mergesort

```
1: function BOTTOM-UP-MERGESORT(K)
2:     for (size ← 1; size < |K|; size ← 2size) do
3:         for (lo ← 1; lo < |K| − size + 1; lo ← lo + 2size) do
4:             mid ← lo + size − 1
5:             hi ← min(lo + 2size − 1, |K|)
6:             MERGE(K, lo, mid, hi)
7:         end for
8:     end for
9: end function
```

Lemma 1. *At the start of kth iteration of outer for loop,* $[i2^{k-1} + 1, min(|K| + 1, (i + 1)2^{k-1} + 1))$ *for* $i \in \{0, \ldots, \frac{n-2^{k-1}}{2^{k-1}}\}$ *constitute a sequence of sorted intervals, each having length* 2^{k-1} *if* $|K|$ *is a power of two. Otherwise, the last run can be smaller than* 2^{k-1}*. Then, after the iteration* $k = \log_2 n$*, the whole sequence* K *is sorted.*

Initialization: At the start of the first iteration of outer for cycle, we have $k = 1$. Hence, the intervals $[i + 1, i + 2)$ for $i \in \{0, \ldots, n - 1\}$ constitute intervals of length $2^{k-1} = 1$, which are trivially sorted.

Iteration: To show that the invariant applies after each iteration, let's assume, that at the start of kth iteration, the intervals $[i2^{k-1} + 1, min(|K| + 1, (i + 1)2^{k-1} + 1))$ for $i \in \{0, \ldots, \frac{n-2^{k-1}}{2^{k-1}}\}$ are sorted and at most $((i+1)2^{k-1}+1)-(i2^{k-1}+1) = 2^{k-1}$ long (with possible exception of the last interval). Then, in the kth iteration, $size = 2^{k-1}$ and MERGE$(K, lo, lo+size-1, min(|K|, lo+2size-1))$ merges intervals $K[lo, lo + 2^{k-1} - 1]$ and $K[lo + 2^{k-1}, min(|K|, lo + 2^k - 1)]$

with $lo = j2^k + 1$ for $j \in \{0, \ldots, \frac{n-2^k}{2^k}\}$. By inserting lo, we get

$$[j2^k + 1, j2^k + 2^{k-1}] \text{ and } [j2^k + 2^{k-1} + 1, min(|K|, j2^k + 2^k)]$$

After termination of the inner for loop, the following intervals are sorted:

$$[j2^k + 1, min(|K|, (j+1)2^k)] \text{ for } j \in \{0, \ldots, \frac{n-2^k}{2^k}\}$$

Then, at the beginning of next iteration, the k is increased and the invariant present in lemma holds.

Termination: During the last iteration, $k = \log_2 n$. By assumption, all intervals of length $2^{\log_2 n - 1}$ are already sorted. Then, after kth iteration, $k = \log_2 n + 1$ and by invariant, interval $[i2^{\log_2 n} + 1, (i+1)2^{\log_2 n} + 1) = [in + 1, in + n + 1)$ for $i \in \{0, \ldots, n - 2^{\log_2 n}\} = \{0\}$ is sorted. Substituting i, we get sorted interval $[1, n + 1)$.

1.6.5 Asymptotic time and space analysis

The recursion of mergesort can be described by:

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n & \text{if } n > 1 \end{cases}$$

In the case of input with size $n = 1$ we have already sorted sequence. Otherwise, the cost $T(n)$ is equal to the cost of sorting $T(\lfloor n/2 \rfloor)$ and $T(\lceil n/2 \rceil)$ combined with the worst case linear cost for merging. The recursion in question can be easily solved by the master theorem, hence the worst case bound is $\mathcal{O}(n \log n)$.

In the best case, we need in each merge at least $\lfloor n/2 \rfloor$ comparisons, that result in $\lfloor n/2 \rfloor$ divide and combine cost. Solving the recursion gives us the cost of comparisons $(1/2)n \log n$ [11, p. 274], which proves the best case time complexity $\Omega(n \log n)$.

The average case complexity has been properly studied and analysed in many articles [24, 25]. The recursion denoting average number of comparisons can be derived by finding out a mean number of elements $\mathbb{E}[S]$, which are placed after one of the runs became empty in the process of merging. Then, at each recursion tree node, at most $n - \mathbb{E}[S]$ comparisons are made. The number of comparisons is then [24, p. 677]:

$$C(n) = C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + n - \mathbb{E}[S] \tag{1.3}$$

Due to the best and worst case bounding of average, it is obvious, that the average case time complexity won't achieve better bound and that the difference can lie only in a negligible lower order terms.

The space complexity is linear in the length of input sequence, due to merging, which is not in-place.

The stability of mergesort is achieved by merging of consecutive runs, which is performed with attention to relative order of equal elements contained in two runs.

1.7 Timsort

Timsort is a fairly new type of hybrid sorting algorithm, which was introduced by Tim Peters in 2002 [26]. It combines a mergesort, binary insertion sort and techniques from McIlroy's "Optimistic Sorting and Information Theoretic Complexity" [27] to achieve more optimal sorting. By being strongly engineered and in many cases the best performer, it found its way into several standard libraries of languages like Python and Java. The most important feature, that makes timsort in many ways superior to other algorithms, is that it is adaptive algorithm.

1.7.1 Adaptive sorting

Adaptive sorting algorithms are algorithms, that can exploit regularities in presorted data without knowing apriori. The more an input instance is presorted, the faster should such an algorithm perform. However, the presortedness can be measured by many measures [28, 29]. In general, to achieve the best running times, the adaptive sorting algorithms should find presortedness using measures, that are asymptotically maximized for the sequence being sorted. Usually, it corresponds to a presorting phase, which costs are included in those of sorting algorithm. Then, by using suitable sorting strategy, it is possible to achieve better running times than comparison-based lower bound $\Omega(n \log n)$.

1.7.2 Runs

The timsort's presortedness measure is based on finding monotonic subsequences in the presorting phase, which is implemented by a simple pass over input sequence K. Within the pass, it compares first two elements to find out a type of ordering, that should be looked for. It can be either non-decreasing or descending. Then, the search continues, until the found run is *minrun* long, or until some element, that violates an arrangement of elements in the so far discovered run is encountered. In both cases, a descending run is immediately reversed in-place, so as to avoid higher merging costs. In addition to reversing, the latter case needs to artificially extend the run to be *minrun* long. This is accomplished by binary insertion sort, which begins by inserting element, that caused the search termination. After constructing a sorted run of length *minrun*,

the same steps are repeated all over again, until all elements are present in some sorted run. The only exception to this rule is the last run, which doesn't have to be exactly *minrun* long. The *minrun* value, that is used to divide K is computed using size $|K|$ on the following principles:

1. *minrun* can't be too big, so that binary insertion sort doesn't become bottleneck

2. *minrun* can't be too small, so that the number of merges is not too high

3. $|K|/minrun$ should be a power of two, or if it is not possible, strictly less than a power of two, so that the last run is equal or close to *minrun* long. The aim of this measure is to trigger merges, which are balanced as much as possible

According to these principles, a suitable *minrun* size was determined to be in the range $[32, 64)$. The *minrun* can be computed by taking first six bits and adding one, if any of the remaining bits are set. A pseudocode, that computes *minrun* looks like:

Algorithm 23 *minrun* computation

```
1: function ComputeMinrun(K)
2:     size ← |K|
3:     l ← 0
4:     while size ⩾ 64 do
5:         l ← l | (size & 1)
6:         size ← size/2
7:     end while
8:     return size + l
9: end function
```



Figure 1.4: The picture depicts a search for non-decreasing and descending runs

1.7.3 Merge Pattern

The important characteristic of timsort is how merging blends with runs discovery. In conflict with the presorting phase naming, both of them should be performed simultaneously to lower space requirements. Each time some run is discovered, its starting and ending indices are pushed to the stack π. Then, the merging can take place, but only when the invariant conditions for the stack are invalidated. Given some arbitrary sequence of independently sorted runs π_i for $i \in \{1, \ldots, \lceil \log_\varphi n \rceil\}$ on top of the stack, the invariant is following:

1. $|\pi_i| > |\pi_{i+1}| + |\pi_{i+2}|$ for $\forall i \in \{1, \ldots, \lceil \log_\varphi n \rceil - 2\}$

2. $|\pi_i| > |\pi_{i+1}|$ for $\forall i \in \{1, \ldots, \lceil \log_\varphi n \rceil - 1\}$

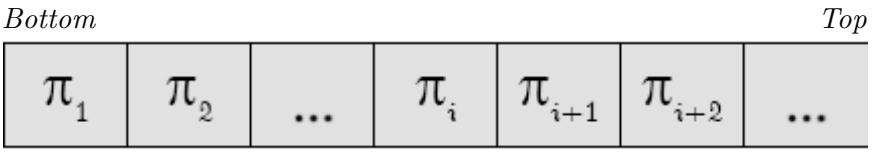


Figure 1.5: Stack π with labeled bottom and top ends.

The second invariant implies, that lengths of runs on the stack constitute a decreasing sequence. The first one on the other hand requires, that runs grow at least as fast as Fibonacci numbers. By a simple observation, the number of entries present on the stack never exceeds logarithm of $|K|$ with the base of golden ratio $\varphi = \frac{1+\sqrt{5}}{2}$ [30, p. 8].

The only possible way, how the invariant can get invalidated is by pushing to the stack. Hence, the invariant invalidation happens each time at the top of the stack. This can be corrected by repeated merging of runs, until the stack eventually complies with the invariant. If it's the first invariant, that is invalidated, the runs to be merged are figured out by comparing $|\pi_i|$ and $|\pi_{i+2}|$. In case of $|\pi_i| < |\pi_{i+2}|$, π_i and π_{i+1} are merged. Otherwise, the merge of two topmost runs π_{i+1} and π_{i+2} is carried out. The same applies to invalidation of the second part of the invariant, and to the case $|\pi_i| = |\pi_{i+2}|$, when the temporal locality plays an important role.

1.7.4 Merging runs

As mentioned in section 1.6.2, the in-place merging is infeasible without additional overhead, that makes it inpractical. However, the memory usage can be optimised to use just $\mathcal{O}(n/2)$ space in the worst case. This can be done by first determining the smaller of runs π_i and π_{i+1}, where $|\pi_i|$ is strictly greater or equal to 32 and $|\pi_{i+1}|$ an arbitrary positive number.

If $|\pi_i| \leqslant |\pi_{i+1}|$, the π_i is copied to a temporary array. Then, the merging from the temporary array and π_{i+1} can place elements starting at first position of π_i. At the start of i*th* element placement with l elements already placed from the temporary array, the common number of free consecutive positions in π_i and π_{i+1} is $(|\pi_i| - l)$. Using induction on this hypothesis, it can be proved, that no additional space is required. Intuitively, the element placing continues until the temporary array is empty. However, in a case when all elements from π_{i+1} have been placed, the remaining elements from the temporary array have to be appended to the back of ordered sequence.

The $|\pi_i| > |\pi_{i+1}|$ case is analogous and π_{i+1} is moved to the temporary array. Subsequently, elements are being stored at the back of π_{i+1} using reversed relational operator.



$\pi_i[1, \ldots, k] \leqslant \pi_{i+1}[1]$ $\qquad$ $\pi_{i+1}[l, \ldots, |\pi_{i+1}|] \geqslant \pi_i[|\pi_i|]$

Figure 1.6: Preliminary searches on two neighbouring runs π_i and π_{i+1}. Elements in gray parts are already in correct position

Another important refinement mentioned in [26], that can considerably speed up sorting are *preliminary searches*. The idea behind preliminary search is, that prior to copying any run to the temporary array, we can check where the first element of run π_{i+1} belongs in sequence π_i. All the elements, that are lower or equal to $\pi_{i+1}[1]$ are already in correct place. Similarly, the preliminary search looks where the last element of π_i belongs in π_{i+1} starting from its last position. In fact, the search itself is implemented using technique called *exponential search* [27, p. 469]. In the first part of preliminary search, the exponential search compares $\pi_{i+1}[1]$ with elements $\pi_i[1]$, $\pi_i[2]$, $\pi_i[4]$, $\pi_i[8]$, increasing by powers of

two, until $\pi_{i+1}[1] < \pi_i[2^k]$. Then, the $\pi_{i+1}[1]$ should be placed to a position in the range $\pi_i[2^{k-1}+1, \ldots, 2^k]$. The number of different positions, that need to be searched is $2^k - (2^{k-1}+1) + 1 = 2^k - 2^{k-1} = 2^{k-1}$. By applying a binary search, $k-1$ steps are sufficient to compute the right spot. All in all, we need $\log_2 |\pi_i| + k - 1 \leqslant 2 \log_2 |\pi_i|$ steps, where $k > 0$ to perform preliminary search on π_i. It can be seen, that the exponential search is in this case superior to the binary search, because it is far more likely, that the found sub-run is small. Still, this refinement may not pay off, but when it does, it saves us many redundant element moves and comparisons.

1.7.5 Galloping mode

Similarly to preliminary searches, the merging of long runs can benefit from using exponential search. The Merge function, that exploits this technique keeps track of how many times in a row an element from certain run has been placed. If this count reaches a constant number (usually 7), the galloping mode is entered. For now, let's assume, that $|\pi_i|$ was smaller or equal to $|\pi_{i+1}|$ prior to Merge call. Then, the galloping is commenced by exponential search for $\pi_i[1]$ in π_{i+1}. After finding a sub-run, that has smaller elements than $\pi_i[1]$, it is appended to the sorted sequence. This move is followed by placing $\pi_i[1]$, that is certainly smaller or equal to the first element in π_{i+1}, if π_{i+1} is non-empty. In the next step, we search π_i where $\pi_{i+1}[1]$ belongs and conclude the iteration by appending $\pi_{i+1}[1]$. The case when $|\pi_i|$ is greater than $|\pi_{i+1}|$ prior to Merge call is analogous and it is commenced by exponential search for $\pi_{i+1}[1]$ in π_i.

The galloping mode is abandoned whenever both exponential searches find sub-runs, that are smaller than boundary constant, that initially triggered a mode change. The merging process then continues in a mode, which in each iteration compares a pair of elements and places one.

1.7.6 Pseudocodes

To correctly understand provided pseudocodes, some notation conventions have to be outlined in advance. First and foremost, the letter π denotes a stack for runs. By using subscripts, π_i denotes ith run. In

Algorithm 24 Timsort

```
1: function TimSort(K)
2:     stack π ← ∅
3:     minrun ← ComputeMinrun(K)
4:     fnrun ← 1
5:     while fnrun ⩽ |K| do
6:         (rstart, rend) ← ComputeRun(K, minrun, fnrun)
7:         PushRun(π, (rstart, rend))
8:         while |π| > 1 do
9:             if |π| > 2 ∧ |π_{|π|-2}| ⩽ |π_{|π|-1}| + |π_{|π|}| then
10:                if |π_{|π|-2}| < |π_{|π|}| then
11:                    merge(π_{|π|-2}, π_{|π|-1})
12:                else
13:                    merge(π_{|π|-1}, π_{|π|})
14:                end if
15:            else if |π_{|π_{i-1}|}| ⩽ |π_{|π|}| then
16:                merge(π_{|π|-1}, π_{|π|})
17:            else
18:                break                ▷ Invariant holds for the stack π
19:            end if
20:        end while
21:        fnrun ← rend + 1
22:    end while
23:    while |π| > 1 do ▷ Merge remaining runs present in the stack
24:        if |π| > 2 ∧ |π_{|π|-2}| < |π_{|π|}| then
25:            merge(π_{|π|-2}, π_{|π|-1})
26:        else
27:            merge(π_{|π|-1}, π_{|π|})
28:        end if
29:    end while
30: end function
```

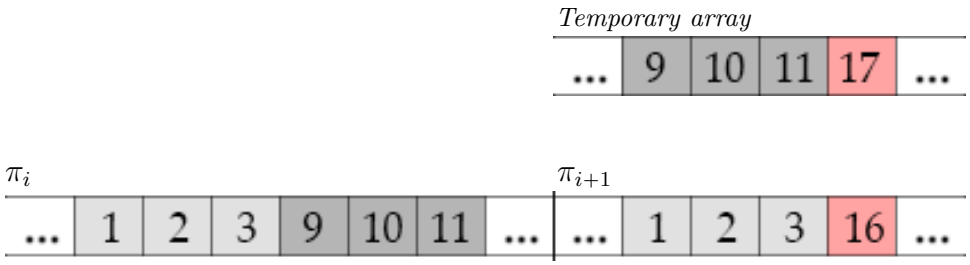


Figure 1.7: One iteration of the galloping mode, that assumes $|\pi_i|$ was smaller or equal to $|\pi_{i+1}|$ prior to MERGE call. The galloping is commenced by a search for all values in π_{i+1}, which are smaller than the first element in temporary array (9). In the next step, the galloping looks for all elements in temporary array, that are smaller or equal to 16. After each search, the found elements are appended to the π_i

like manner, $\pi_{|\pi|}$ denotes a run on top of the stack π and $\pi_{|\pi|-1}$ one underneath it. For the size of stack or run, we utilise the absolute value notation. Lastly, to access jth element of ith run, we use array indexing $\pi_i[j]$.

The timsort's pseudocode proceeds as follows. As a first step, it initializes the stack π with an empty set of index pairs. Following stack initialization, the *minrun* is set to ComputeMinrun return value. Subsequently, the initialization phase is concluded by setting $fnrun$ to one. Then, in each iteration of the outer while loop, a call to ComputeRun returns a run-limiting pair of indices ($rstart$, $rend$), that are pushed to the stack π. However, the action of pushing may invalidate the invariant and trigger merging, that tries to restore it in the nested while cycle. Last, but not least, each iteration of the outer while loop is independently on invariant invalidation finished by setting $fnrun$ to one past $rend$. Then, the search for sorted run in next iteration begins with correctly increased $fnrun$ index. By assuming that the input sequence K is finite, the $fnrun$ will eventually cause a termination on $fnrun > |K|$. Following while loop termination, there's still a possibility, that the stack π contains more than one run, that need to be collapsed. In particular, this happens when the last run wasn't exactly *minrun* long and after its push, the invariant was preserved. Timsort then terminates by merging on top of the stack, each time

restoring the invariant or otherwise merging a two topmost runs, until there's only one sorted run, which constitutes sorted K.

Algorithm 25 Compute sorted run

Require: $fnrun$ holds the lowest index not present in any run

```
1: function ComputeRun(K, minrun, fnrun)
2:     rend ←min(fnrun + minrun − 1, |K|)
3:     fnsorted ←FindFnSorted(K, fnrun, rend)
4:     if (K[fnrun, ..., fnsorted − 1] is in descending order) then
5:         Reverse(K, fnrun, fnsorted − 1)
6:     end if
7:     BinaryInsertsort(K, fnrun, fnsorted, rend)
8:     return(fnrun, rend)
9: end function
```

To clarify how runs are being computed, the following steps are carried out in each ComputeRun, assuming, that $fnrun$, which denotes the first element of run to be pushed has a correct value:

1. The variable $rend$ is initialized with correct ending index of the run

2. The variable $fnsorted$ is initialized by FindFnSorted return value, which is the smallest index of the element in range $[fnrun, \ldots, rend]$, that is out of order. The search itself is performed in linear time.

3. If the sorted subrun $K[fnrun, \ldots, fnsorted - 1]$ is in descending order, it is reversed

4. Given indices $fnrun$, $fnsorted$ and $rend$, the binary insertion sort insertsorts $K[fnsorted, \ldots, rend]$ into $K[fnrun, \ldots, fnsorted - 1]$

5. The pair of indices ($fnrun$, $rend$) determining lower and upper run border is returned

1.7.7 Asymptotic time and space analysis

The worst case occurs when K is a truly random sequence. In all such cases, the complexity is $\mathcal{O}(n \log n)$ [26]. The worst case cost $T(N)$ of the sorting can be also derived by summing up a number of steps required to precompute sorted runs and to merge them. In case of input instance of length N, we have $\lceil N/minrun \rceil$ runs, each of which is presorted by binary insertion sort. Using operational interpretation of inversions measure [28, p. 155], we can derive a number of element moves performed in one sorting. The cost of presorting $S(N)$ is:

$$S(N) = \left\lceil \frac{N}{k} \right\rceil \frac{3k}{2} + \sum_{i=1}^{\lceil N/k \rceil} (Inv(K[(i-1)k+1, \ldots, min(ik, N)]) + k - 2)$$

where $k = minrun$. The first term in the summation corresponds to a number of swaps, that need to be performed in a run of length k. The second term, specifically $k-2$ denotes the least number of swaps required in sorting one run. Lastly, we have to count in the worst case costs for run reversals, what amounts to additional $\lceil N/k \rceil (3k/2)$ if one swap consists of three element moves.

The complexity of merging part can be derived by observing, that after the ith push to the stack, the number of merges that is triggered equals to the count of trailing zeroes of i in the binary representation. Seemingly, each of these subsequent merges, merges runs of length larger by a factor of two. Thus, the merging cost $M(N)$ is:

$$M(N) = \sum_{i=1}^{\lceil N/k \rceil} \sum_{j=1}^{ctz(i)} k2^j$$

Combining $S(N)$ and $M(N)$, we get:

$$T(N) = \sum_{i=1}^{\lceil N/k \rceil} (Inv(K[(i-1)k+1, \ldots, min(ik, N)]) + k - 2) + \left\lceil \frac{N}{k} \right\rceil \frac{3k}{2} + \sum_{i=1}^{\lceil N/k \rceil} \sum_{j=1}^{ctz(i)} k2^j$$

The best case complexity has seen some serious improvement due to the adaptive nature of timsort. In case of already sorted input,

the presorting of runs costs just linear number of comparisons and movements incurred by merging are prevented by preliminary searches. Thus, the best case time complexity is $\mathcal{O}(n)$.

The average case time complexity is more complex to derive. Since there's no relevant article or publication, that would derive the average case bound, we will try to approximate it in the rest of this section. For that purpose, we can use similar arguments used in the insertion sort and mergesort average case analysis [24]. The average cost of the presorting phase greatly depends on a mean value of already sorted elements in each run. Trivially, the base case of two elements constitutes a non-decreasing or descending sequence with certainty. Overall, the probability is governed by the following function:

$$P(R = r) = 2\binom{n + r - 1}{r}\left(\frac{1}{n}\right)^r$$

where R is a random variable denoting a length of presorted subsequence from the range $[1, 64)$ and n a number of unique values. Substituting $\mathbb{E}[R]$ into cost of the presorting phase $S_{AV}(N)$:

$$S_{AV}(N) = \frac{3N\mathbb{E}[R]}{4k} + \sum_{i=1}^{\lceil N/k \rceil} \sum_{j=\mathbb{E}[R]+1}^{k} \frac{j-1}{2}$$

The approach to $S_{AV}(N)$ summation is nearly identical to the worst case one. The first term within the equation represents the average number of element moves required to reverse a descending part of runs. The second term on the other hand computes the average number of moves performed to sort all runs. In order to give a good estimate on average cost of the merging phase, we need to compute a mean of random variable L, that denotes a number of elements, which remained, after all elements of one run have been already placed. Such a mean value can be computed by the summation:

$$\mathbb{E}_t[L] = \sum_{l=1}^{t} lP(L = l)$$

where t is a size of each run, that is being merged and:

$$P_t(L = l) = \frac{\binom{2t-l}{t}}{\binom{2t}{t}} + \frac{\binom{2t-l}{2t}}{\binom{2t}{t}}$$

is the probability, that one of the runs has remaining l elements, after the other one became empty. Now, the average cost of merging is following:

$$M_{AV}(N) = \sum_{i=1}^{\lceil N/k \rceil} \sum_{j=1}^{ctz(i)} (k2^j - \mathbb{E}_{k2^{j-1}}[L])$$

Combining the average case cost of the presorting and merging phase, we get:

$$T_{AV}(N) = \frac{3N\mathbb{E}[R]}{4k} + \sum_{i=1}^{\lceil N/k \rceil} \sum_{j=\mathbb{E}[R]+1}^{k} \left(\frac{j-1}{2}\right) + \sum_{i=1}^{\lceil N/k \rceil} \sum_{j=1}^{ctz(i)} (k2^j - \mathbb{E}_{k2^{j-1}}[L])$$

It's also important to note, that $T_{AV}(N)$ assumes, that merging is done on runs, that are equal in size. This is no problem at all and the summation should provide a very good approximation for the average case, although not that tight, when the last run is not exactly *minrun* long.

The space complexity is determined by the stack and merging needs. The logarithmic space required by the stack π is in this case only negligible amount, because a major linear cost of $\lceil N/2 \rceil$ is used by the merging function. Hence, the timsort has $\mathcal{O}(n)$ space complexity.

The timsort's stability is achieved by presorting and merging, which are performed with respect to the stability property. For the most part, this involves correct use of relational operators.

2 Performance analysis

2.1 Implementation

All algorithms described in above chapters were also implemented in *C++*. They feature a common interface, that accepts on input two *RandomAccessIterator*s and a comparator. Similarly to the standard library functions, the first iterator points to the beginning and the other one, one past the end of sequence to be sorted. Consequently, this provides a convenient way how to store them and perform automated benchmarking. In addition to common interface, all pseudocodes were converted and implemented in their iterative version in order to prevent stack overflows on larger instances. The following list specifies distinct versions of algorithms, that were implemented and subsequently benchmarked:

- Randomized bogosort
- Stooge sort
- Insertion sort
- Binary insertion sort
- Bottom-up heapsort
- Bottom-up mergesort
- Quicksort with last element pivot
- Quicksort with random pivot
- Quicksort with median of three pivot
- Timsort

Implementations itself were compiled using *clang++* version 3.9 and `-std=c++1z -stdlib=libstdc++ -O3 -Wall -Wextra -pedantic` parameters. The selected version of *C++* conforms to 2017 Draft International Standard. The compilation was done on Linux system with libstdc++ standard library used by default. All switches, that alter the

compilation process were included in CMakeLists.txt file, that specifies *benchmark* compilation target for the CMake build system.

2.2 Testing environment

The benchmarking was carried out on Dell Inspiron 7537 laptop with Intel Core i7-4500u processor, that has maximum 3GHz frequency, 32K L1 cache, 256K L2 cache and 4096K L3 cache. Moreover, during benchmarking, there was at disposal 16GB DDR3L 1600MHz RAM. The operating system, on which tests ran was Ubuntu 16.04 with kernel version 4.4.0-59-generic.

As with other operating systems, processes don't have exclusive access to processing unit and Linux system is no exception. The processor on which certain process will run ultimately depends on balancing system and soft affinity, if its processing unit was already chosen. As a result, all units are evenly utilised. In general, this applies also to the interrupts from drivers and devices. All these undesirable factors cause, that measurings are less accurate.

In the performance analysis, we will try to avoid such factors as much as possible. The first countermeasure involves *init* process, that launches all other tasks, which in turn inherit *init*'s *cpus_allowed* mask. Intuitively, by changing its cpu mask prior to its launch, we can restrict the set of processing units, on which all processes can run. This can be done by explicitely specifying proper *isolcpus* kernel boot parameter, that isolates CPUs from kernel scheduler [31]:

```
isolcpus = cpu_number,[cpu_number,...]
```

Actually, this is not enough, because after launching benchmarks, its process would have received *init*'s mask. For this purpose, the *sched.h* header provides the function `sched_setaffinity` [32], that can adjust process's hard affinity. Secondly, we can restrict the set of cpus, that will handle interrupts triggered by drivers and devices. A good insight of interrupt handling provides a file `/proc/interrupts`, that records a number of interrupts handled by each processing unit from certain IRQ line. The set of processors can be temporarily restricted by changing hexadecimal values in `/proc/irq/X/smp_affinity`, where `X` is a number of IRQ line and by modifying `/proc/irq/default_smp_affinity`, that is used to set a default affinity for new interrupt lines [33].

Lastly, to remove dependency on performance of underlying hardware, the swap space was turned off during all measurings. This countermeasure is primarily focused on more space-consuming algorithms, that could possibly manifest unexpectedly bad times on large inputs.

2.3 Data generation

In general, the data generation is of great importance in determining performance of sorting algorithms. In this performance analysis, to produce nearly unique instances, we exploit randomness by using mersenne twister pseudorandom number generator in combination with uniform distribution class present in the standard library. All in all, we have tested algorithms with the following sequence types:

- **Sorted:** Even though the occurance of sorted sequence is unlikely, each reasonable sorting algorithm should behave optimally and terminate quickly. The generation of sorted sequences is done as follows. First, let's assume that the underlying element type has w bits, that can represent at most 2^w distinct elements. Then, by dividing 2^w by the size of sequence, that needs to be generated, we get a value l, that denotes a maximum amount by which two consecutive elements can differ. Following l computation, the loop over sequence to be returned is commenced. In the first iteration, the value pushed back to the sequence is random number generated from the range $[0, \ldots, l]$. All remaining iterations do the same, except that the number pushed is summation of the previous one and the random one from the range $[0, \ldots, l]$.

- **Reverse sorted:** The reverse sorted input poses same challenges to sorting algorithms as the sorted one. The generating of reverse sorted sequence is carried out identically, apart from that values are inserted from the back to front.

- **Nearly sorted:** Nearly sorted sequences are obtained from sorted sequences by randomly choosing pairs of indices and swapping at most 8% of elements.

- **Nearly reverse sorted:** Nearly reverse sorted sequences are obtained from reverse sorted sequence by swapping at most 8% of all elements.

- **Random:** The random sequences are generated using mersenne twister pseudorandom number generator and uniform distribution class, that ensures, that all generated values appear with the same probability.

- **K-monotone:** The K-monotone sequence denotes a sequence of alternating sorted and reverse sorted subsequences of length K. In most sequences encountered, the length of all monotone subsequences is very low. Therefore, for the performance analysis, we have used the K equal to 2, 4 and 32.

- **K-shuffled:** The K-shuffled sequence denotes input, which is obtained from some sorted sequence by randomly shuffling each successive K elements. This causes displacement of each element by at most K. Hence, it can detect how the sorting algorithms can cope with smaller number of inversions, that are present only in separate K-tuples. In the benchmarking source code, the K was gradually set to 16, 32 and 64.

- **K-restricted:** The K-restricted sequence is a sequence, that contains only constant number of unique elements. For some sorting algorithms, this may trigger the worst case complexity, hence, it is a suitable test to look into. The number of distinct elements $K(n)$ is computed by:

$$K(n) \quad \begin{cases} \lfloor 0.08n + 1 \rfloor & \text{for } n \leqslant 2^6 \\ \lfloor \log_2 n \rfloor & \text{for } n > 2^6 \end{cases}$$

  where n is a size of sequence to be generated. The order in which they appear in sequence is random and each permutation is equally probable.

For the underlying element type, we have used `uint_fast32_t` and `string`. The `uint_fast32_t` element type is the fastest unsigned integer type with the width of at least 32 bits, that is also produced by mersenne

twister by default. The `std::string` type was chosen to test sorting on larger objects, that require even more comparisons. By benchmarking on strings of proper length, it should be observable, that sorting algorithms are substantially slower, if their complexity is largely constituted by comparison costs.

Another issue arises, when generating *sorted* or *reverse sorted* sequences of strings. The random generation and subsequent sorting could depending on size take quite a long time to produce a single sequence. Therefore, we chose another approach. First, we generate one random sequence, from which all the remaining strings are derived, by increasing the rightmost character. In case it exceeds 'z', it is set to 'a' and a character to the left is increased by one. However, this approach is error prone, because there may not be enough permutations greater than the initial one. This is tricky, but it can be easily avoided. Let's assume, that the set L denotes a set of lowercase characters from 'a' through 'z', that are being used in strings. With the set size $|L|$, we compute $t = \lceil \log_{|L|}(size + 1) \rceil$, that denotes a number of character positions required to create *size* permutations. Then, in the initially generated string p of length *len* (assuming $|L|^{len} > size$), we have to decrease character $p[len - t]$ by one, if it is equal to 'z'. As can be seen, this improvement ensures, that we have enough space to generate required number of permutations.

To store all sequences, the implementation uses `std::vector` container. The vector was chosen in particular for its amortized complexity and elements stored in one continuous chunk of memory.

2.4 Interface

The benchmarking code used to evaluate sorting algorithms features a simple interactive interface, that provides a handful of commands with options, that can setup the following parameters:

- **Sorting algorithms:** The interface allows to pick any subset of sorting algorithms, that will be tested. By default, it tests all algorithms.

- **Sequence types:** The interface provides option to test any subset of sequence types. By default, it tests all types of sequences.

- **Element types:** As mentioned above, the benchmarking can use two element types, the `uint_fast32_t` and `std::string`. By default, it uses both of them.

- **Sequence size and repeats:** The sequence size and repeats are provided by four numbers, namely the *min* (minimum size), *step*, *max* (maximum size) and *rpt* (number of repetitions). In benchmarking, each triplet of size, sequence and element type is sorted exactly *rpt* times by each sorting algorithm. The sequence sizes are computed by increasing *min* by *step*, until *max* is exceeded.

Moreover, it has also `help` command, that prints extensive description similar to the bash manual pages, which explains the use of benchmarking tool. The tests itself are started by the `run` command.

2.5 Data evaluation

All the time measurings were done by the `chrono::steady_clock` class from the standard library, that reliably measures intervals [34]. The data generation ran on the fly, so that it wouldn't consume additional hard drive space. It didn't influence the time measurings at all, because timestamps were taken right before and after calling each sorting algorithm. The benchmarking ran for each introduced element and sequence type. However, due to polynomial differences in the worst case complexities, the testing had to be split into three parts. The first part tested only bogosort, that has complexity growing faster than exponentially. The second part tested all the quadratic average time solutions and quicksort with last element pivot, that performs badly on sorted or reverse sorted sequences. Lastly, the third part tested all the remaining algorithms with $\mathcal{O}(n \log n)$ worst case complexity and quicksorts with random and median of three pivot.

All of the computed average times in seconds were written to the separate CSV files, that are named based on the sequence and element type. For example, the *Sorted_uint* file holds the results for sorted sequences of `uint_fast32_t`. The recorded data present in separate CSV files were processed in Microsoft Excel.

2.5.1 The first part of measurings

In the first part, we have tested bogosort at input sizes ranging from 5 to 12, with 6 repeats for each size and pair of sequence and element type. Based on measured times, bogosort performed equally bad on all sequence types, except for the *sorted*, *krestricted* and the *32monotone* type. On the afore-mentioned input sizes, the *sorted* and *32monotone* sequence types were trivially sorted. On all such sequences bogosort terminated immediately after one pass. For the *krestricted* type, the number of possible permutations was lowered by the fact, that only $\lfloor 0.08n + 1 \rfloor$ unique elements were present in each sequence. Thus, it was very likely to terminate earlier. On all other sequences longer than 11, the sorting took inordinate amount of time. Specifically, some instances of size 12 took even hundreds of seconds to randomly generate a sorted sequence.

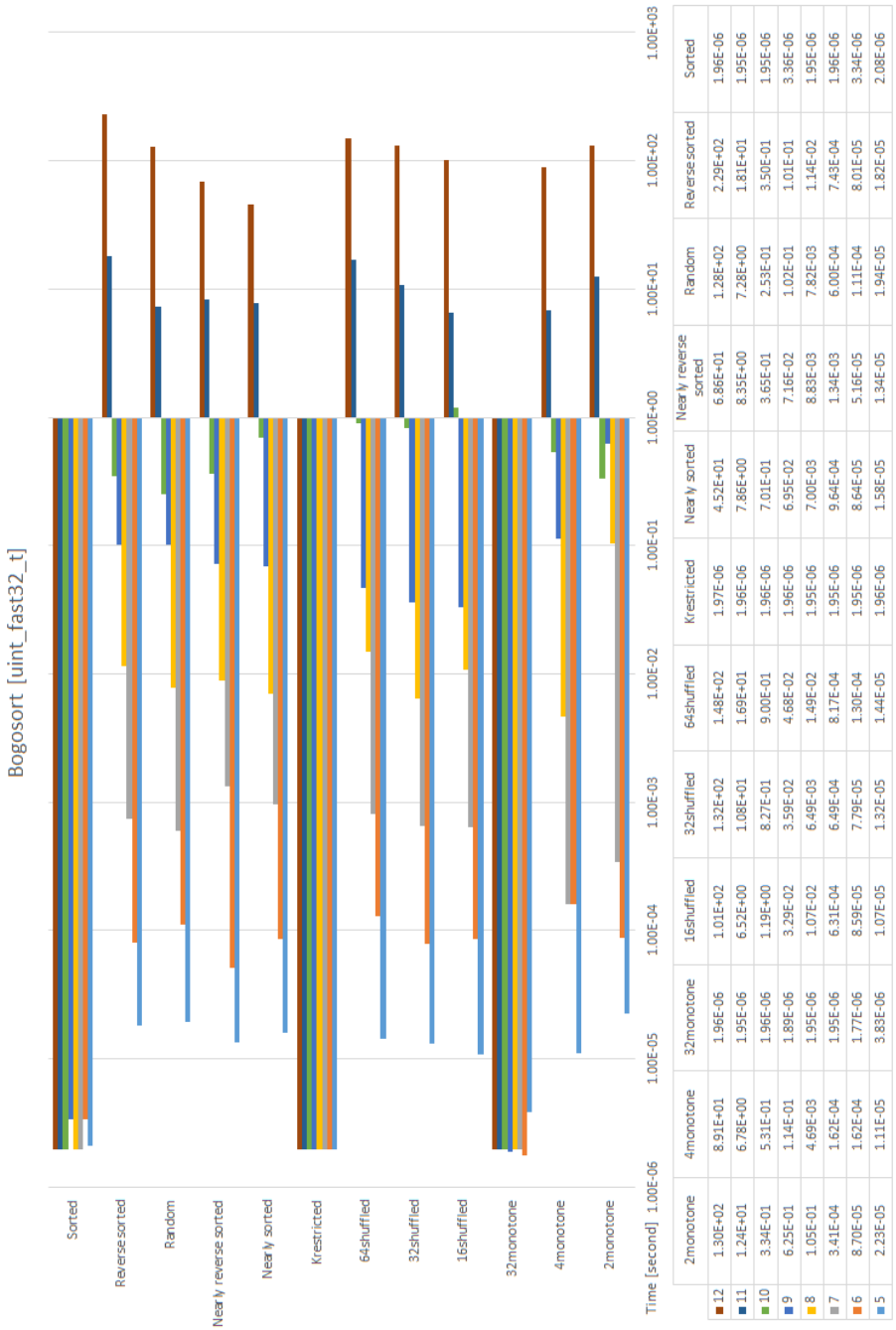
From the measured times, it is apparent, that bogosort's performance is only dependent on a number of unique element values and a size of sequence being sorted. As a result, we can conclude, that presortedness of any kind (except for a complete one) has no impact on its performance and that it is a good example of pessimal sorting algorithm.

2.5.2 The second part of measurings

In the second part, we have tested insertion sort, binary insertion sort, stooge sort and quicksort with last element pivot. Tested sequence sizes were dependent on element type. For `uint_fast32_t`, the tested sizes were multiples of $200,000$, from $20,000$ up to $1,020,000$. Because of excessive execution times required by some sorting algorithms, the sequence sizes of `string`s of length 32 were only multiples of $100,000$, from $20,000$ up to $420,000$. The measurings were repeated twice for each size, element and sequence type.

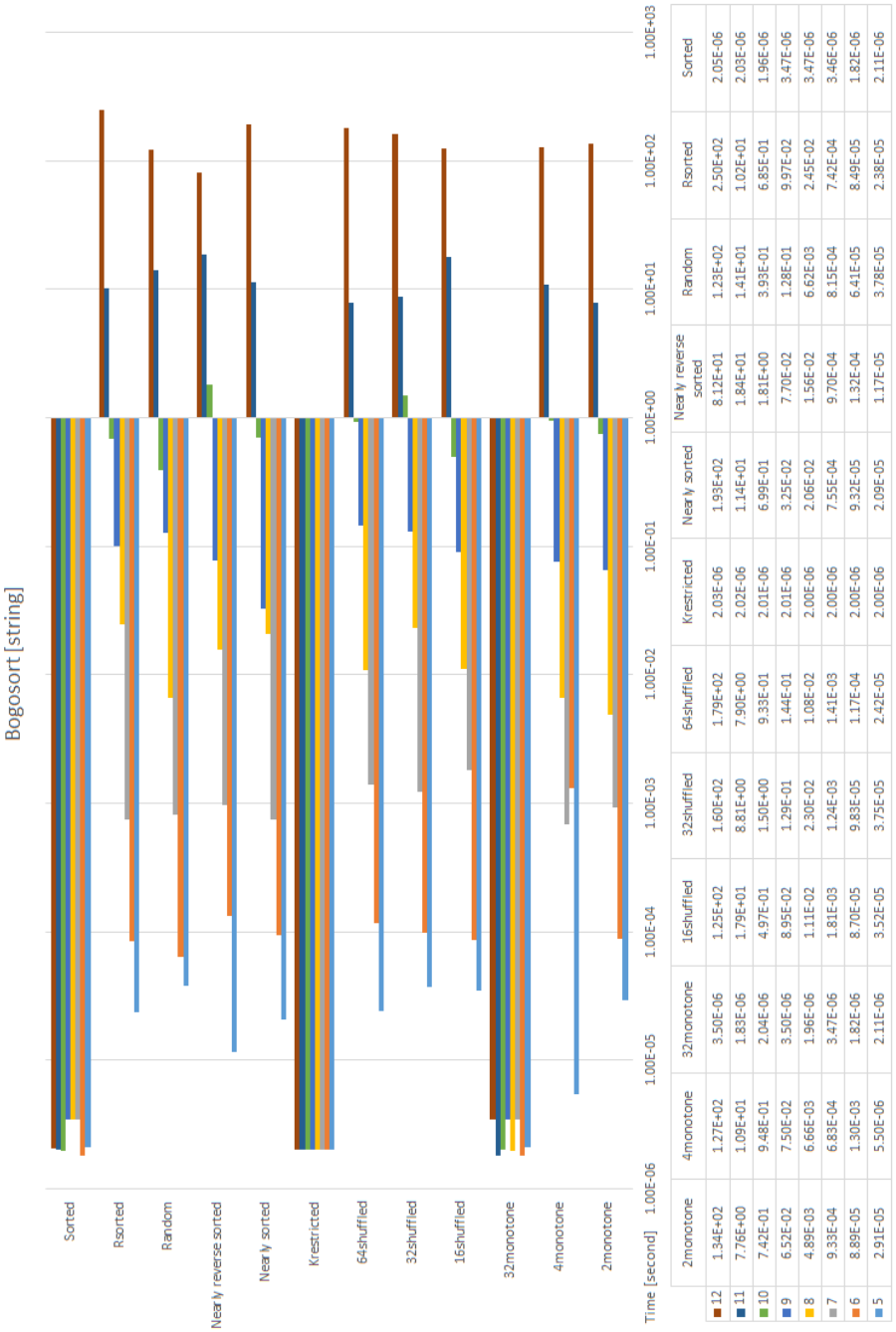
From among all sorting algorithms tested in the second part, the worst performing one was stooge sort. On sequences of length $20,000$, it took nearly 1000 seconds for `uint_fast32_t` type and astonishing one hour for `string`s. Hence, it wasn't possible to measure time on even larger sequences.

On all sequence types, binary insertion sort dominated in comparison to regular insertion sort, largely due to its subquadratic number of



Time [second]	2monotone	4monotone	32monotone	16shuffled	32shuffled	64shuffled	Krestricted	Nearly sorted	Nearly reverse sorted	Random	Reverse sorted	Sorted
12	1.30E+02	8.91E+01	1.96E-06	1.01E+02	1.32E+02	1.48E+02	1.97E-06	4.52E+01	6.86E+01	1.28E+02	2.29E+02	1.96E-06
11	1.24E+01	6.78E+00	1.95E-06	6.52E+00	1.08E+01	1.69E+01	1.96E-06	7.86E+00	8.35E+00	7.28E+00	1.81E+01	1.95E-06
10	3.34E-01	5.31E-01	1.96E-06	1.19E+00	8.27E-01	9.00E-01	1.96E-06	7.01E-01	3.65E-01	2.53E-01	3.50E-01	1.95E-06
9	6.25E-01	1.14E-01	1.89E-06	3.29E-02	3.59E-02	4.68E-02	1.96E-06	6.95E-02	7.16E-02	1.02E-01	1.01E-01	3.36E-06
8	1.05E-01	4.69E-03	1.95E-06	1.07E-02	6.49E-03	1.49E-02	1.95E-06	7.00E-03	8.83E-03	7.82E-03	1.14E-02	1.95E-06
7	3.41E-04	1.62E-04	1.95E-06	6.31E-04	6.49E-04	8.17E-04	1.95E-06	9.64E-04	1.34E-03	6.00E-04	7.43E-04	1.96E-06
6	8.70E-05	1.62E-04	1.77E-06	8.59E-05	7.79E-05	1.30E-04	1.95E-06	8.64E-05	5.16E-05	1.11E-04	8.01E-05	3.34E-06
5	2.23E-05	1.11E-05	3.83E-06	1.07E-05	1.32E-05	1.44E-05	1.96E-06	1.58E-05	1.34E-05	1.94E-05	1.82E-05	2.08E-06

Figure 2.1: Bogosort's average times on sequences of `uint_fast32_t`



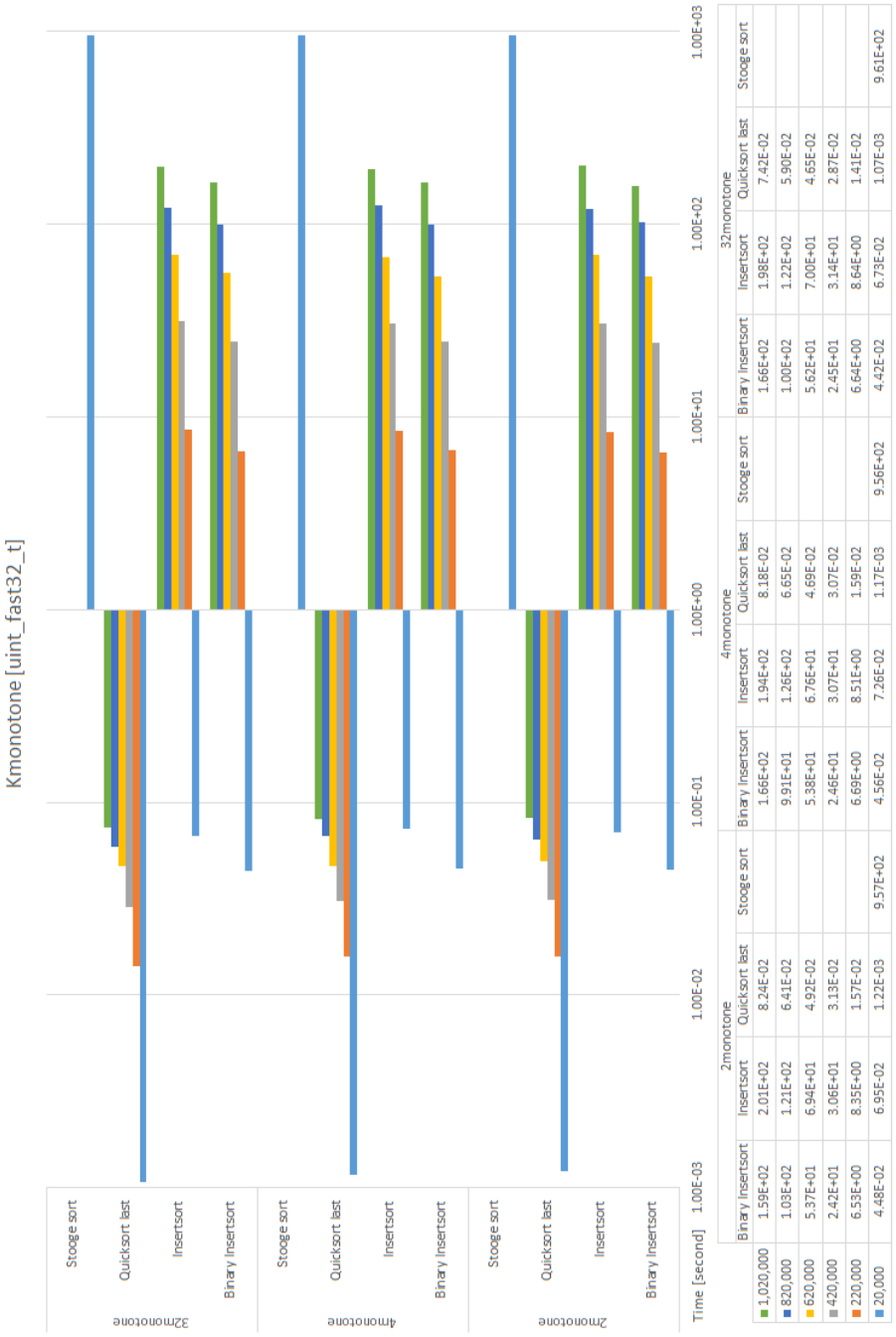
	2monotone	4monotone	32monotone	16shuffled	32shuffled	64shuffled	Krestricted	Nearly sorted	Nearly reverse sorted	Random	Rsorted	Sorted
12	1.34E+02	1.27E+02	3.50E-06	1.25E+02	1.60E+02	1.79E+02	2.03E-06	1.93E+02	8.12E+01	1.23E+02	2.50E+02	2.05E-06
11	7.76E+00	1.09E+01	1.83E-06	1.79E+01	8.81E+00	7.90E+00	2.02E-06	1.14E+01	1.84E+01	1.41E+01	1.02E+01	2.03E-06
10	7.42E-01	9.48E-01	2.04E-06	4.97E-01	1.50E+00	9.33E-01	2.01E-06	6.99E-01	1.81E+00	3.93E-01	6.85E-01	1.96E-06
9	6.52E-02	7.50E-02	3.50E-06	8.95E-02	1.29E-01	1.44E-01	2.01E-06	3.25E-02	7.70E-02	1.28E-01	9.97E-02	3.47E-06
8	4.89E-03	6.66E-03	1.96E-06	1.11E-02	2.30E-02	1.08E-02	2.00E-06	2.06E-02	1.56E-02	6.62E-03	2.45E-02	3.47E-06
7	9.33E-04	6.83E-04	3.47E-06	1.81E-03	1.24E-03	1.41E-03	2.00E-06	7.55E-04	9.70E-04	8.15E-04	7.42E-04	3.46E-06
6	8.89E-05	1.30E-03	1.82E-06	8.70E-05	9.83E-05	1.17E-04	2.00E-06	9.32E-05	1.32E-04	6.41E-05	8.49E-05	1.82E-06
5	2.91E-05	5.50E-06	2.11E-06	3.52E-05	3.75E-05	2.42E-05	2.00E-06	2.09E-05	1.17E-05	3.78E-05	2.38E-05	2.11E-06

Figure 2.2: Bogosort's average times on sequences of `strings`, that are 32 letters long

comparisons. However, it wasn't true for *sorted* and *kshuffled* sequences of all lengths. On *sorted* sequences, insertion sort performed better, because it carried out constant number of comparisons in each iteration. Overall, it made about $\log n$ factor less comparisons than binary insertion sort. In addition, very minor differences favouring insertion sort over binary insertion sort were recorded on the *kshuffled* sequences. A major factor was a number of comparisons made on average, where insertion sort performed just $\mathcal{O}(\frac{Kn}{2})$ comparisons, while binary insertion sort at most $\mathcal{O}(n \log n)$. However, for the K close to 1048 or greater, a number of inversions was sufficiently high, that binary insertion sort outperformed insertion sort. Inversions played even greater role in *kmonotone*, *krestricted*, *nearly reverse sorted*, *random* and *reverse sorted* sequences of strings, where a difference in number of comparisons manifested even more in measured times.

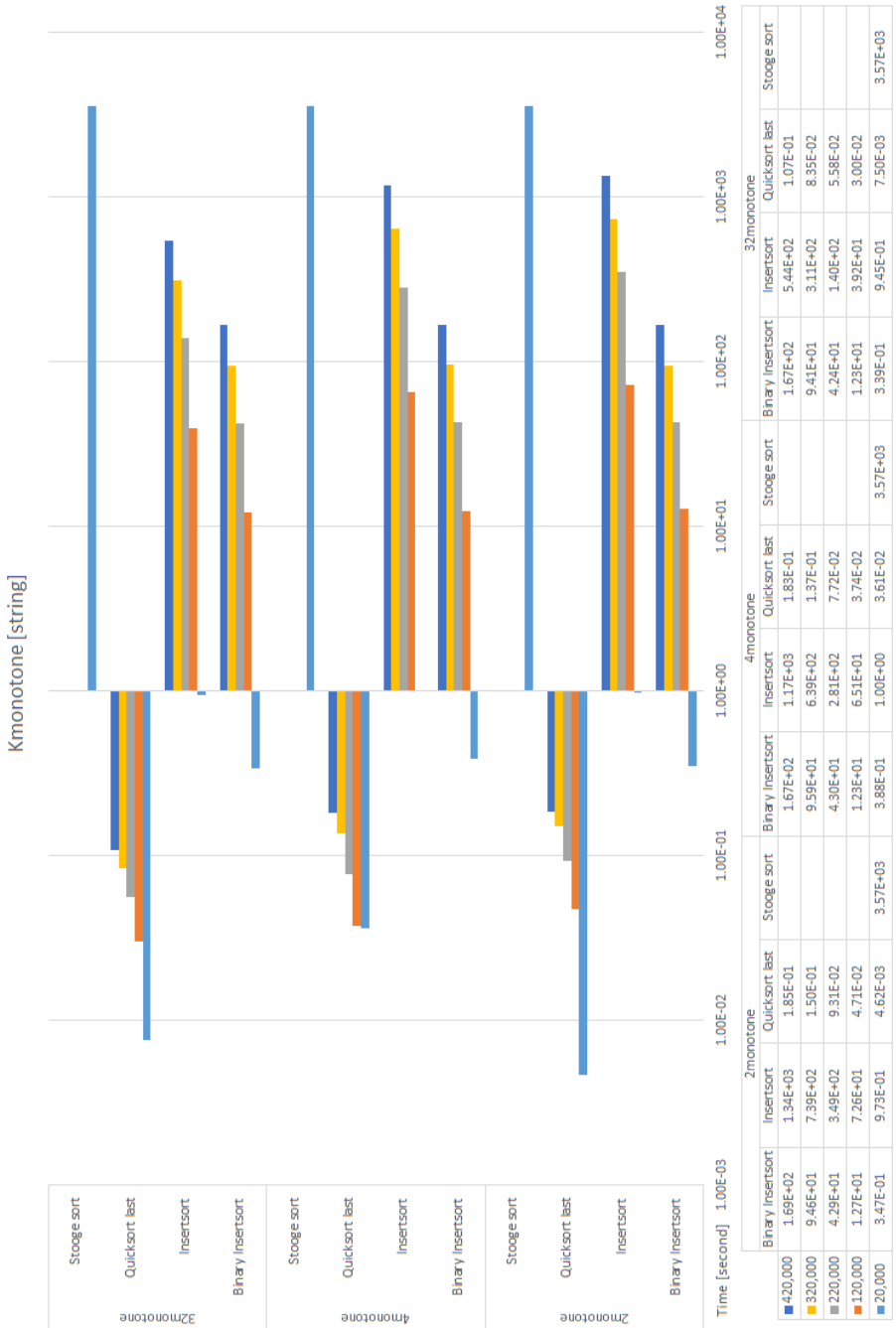
The quicksort with last element pivot considerably outperformed insertion sort and its binary variant on all inputs, except for the *sorted* and *kshuffled* sequence types. In general, the *sorted* and *reverse sorted* sequences are well-known quicksort's worst case inputs, that trigger unbalanced partitioning and linear recursion depth. Our measurings back up such assumption and on the *sorted* sequence type, the quicksort was slower than both insertion sorts. On the other hand, the quicksort performed better on the *reverse sorted* sequences of `uint_fast32_t`, primarily because of linear number of swaps. However, the testing also discovered, that it is slower than binary insertion sort on `string`s of length 32. The reason, that may explain the results is, that quicksort's summary costs for comparisons outbalanced the quadratic swap costs of binary insertion sort.

The *kshuffled* sequences form a nearly sorted input, in which all elements in each K long subsequence are greater or equal to all elements before. This presortedness causes rather unbalanced partitioning for the quicksort, making it at least ten times slower than binary insertion sort. However, for the growing K, the probability of completely bad partitioning becomes very unlikely. Hence, it can still behave acceptably.



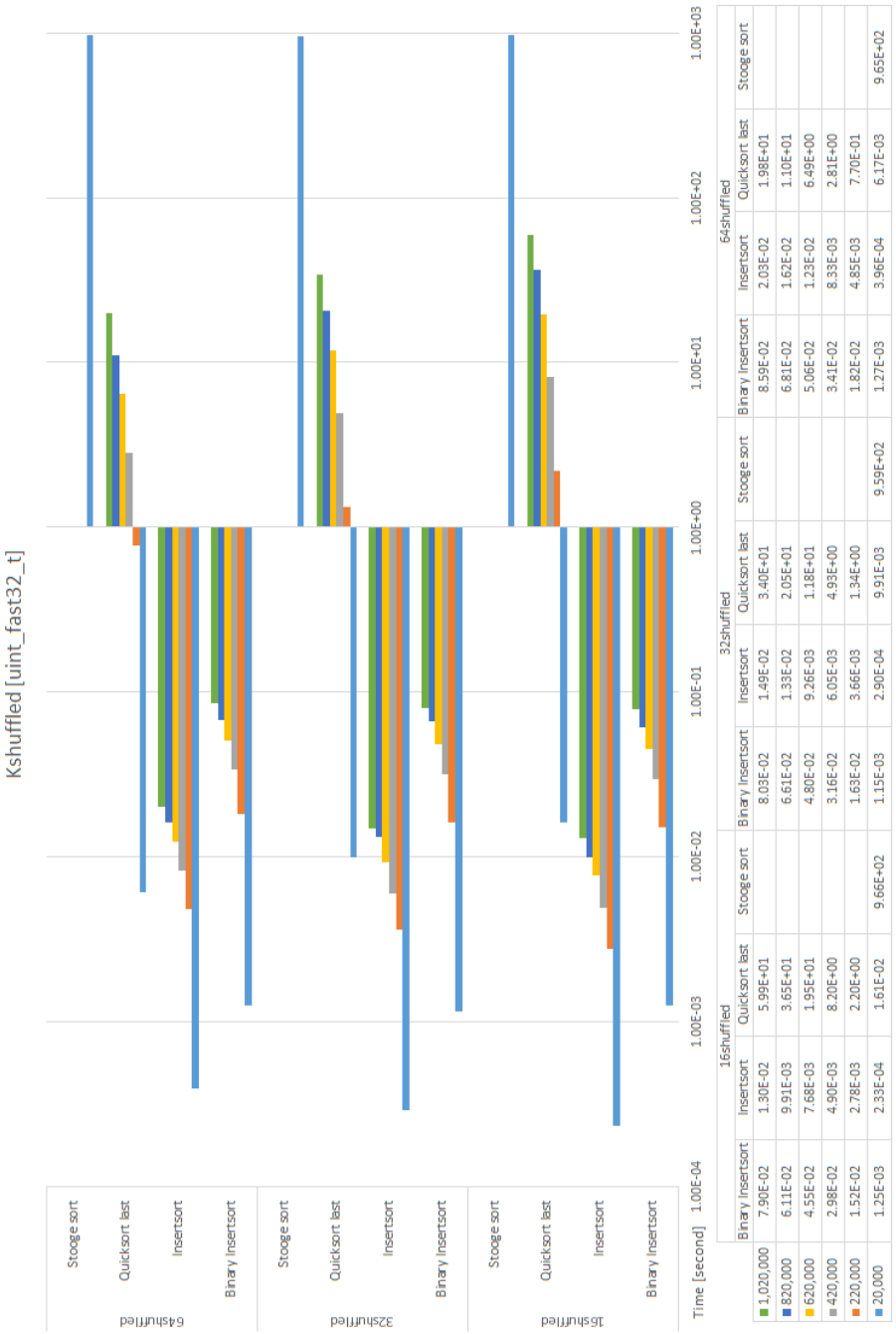
	2monotone				4monotone				32monotone			
	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort
1,020,000	1.59E+02	2.01E+02	8.24E-02		1.66E+02	1.94E+02	8.18E-02		1.66E+02	1.98E+02	7.42E-02	
820,000	1.03E+02	1.21E+02	6.41E-02		9.91E+01	1.26E+02	6.65E-02		1.00E+02	1.22E+02	5.90E-02	
620,000	5.37E+01	6.94E+01	4.92E-02		5.38E+01	6.76E+01	4.69E-02		5.62E+01	7.00E+01	4.65E-02	
420,000	2.42E+01	3.06E+01	3.13E-02		2.46E+01	3.07E+01	3.07E-02		2.45E+01	3.14E+01	2.87E-02	
220,000	6.53E+00	8.35E+00	1.57E-02		6.69E+00	8.51E+00	1.59E-02		6.64E+00	8.64E+00	1.41E-02	
20,000	4.48E-02	6.95E-02	1.22E-03	9.57E+02	4.56E-02	7.26E-02	1.17E-03	9.56E+02	4.42E-02	6.73E-02	1.07E-03	9.61E+02

Figure 2.3: Average times on *kmonotone* sequences of `uint_fast32_t`



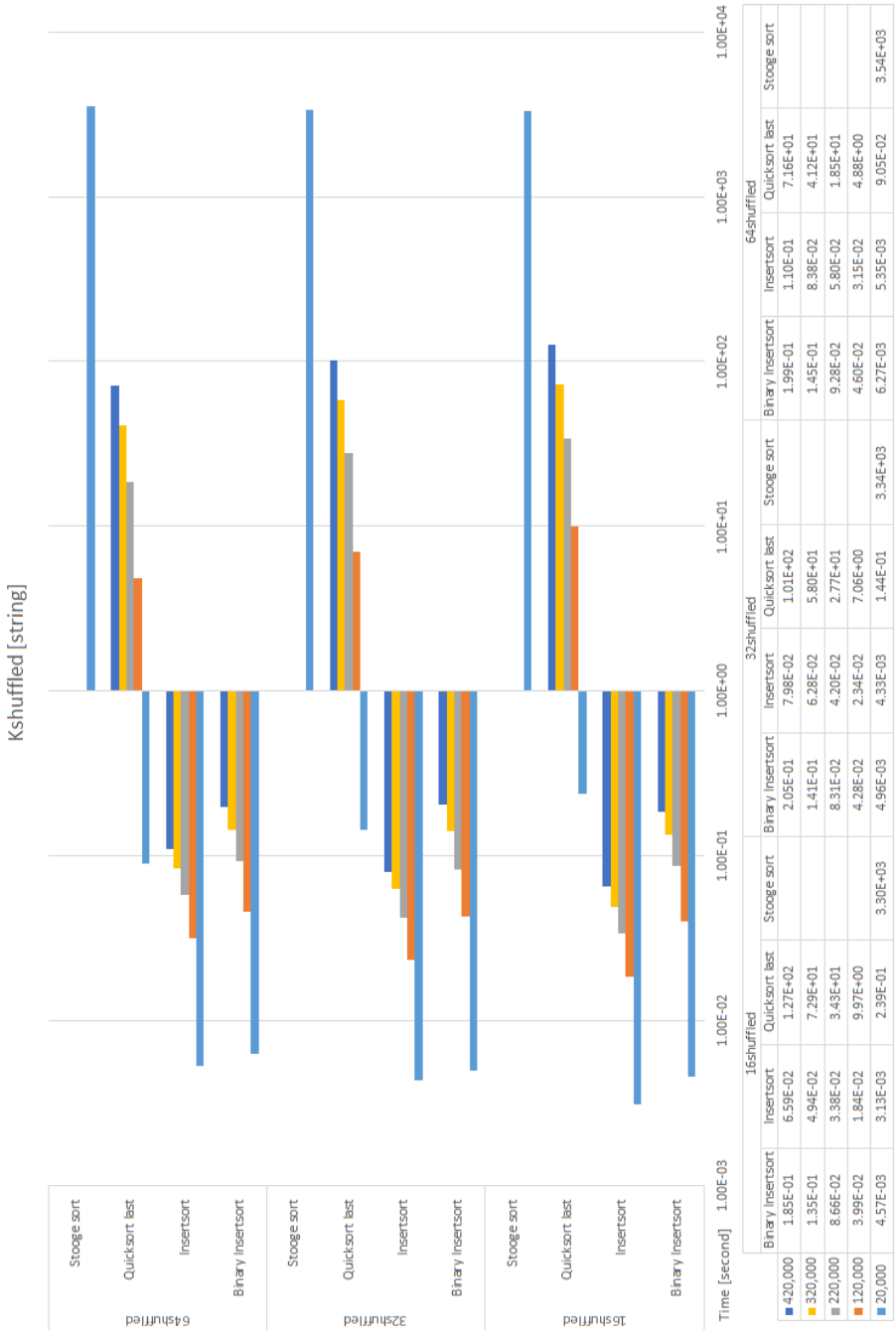
	2monotone				4monotone				32monotone			
	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort
420,000	1.69E+02	1.34E+03	1.85E-01		1.67E+02	1.17E+03	1.83E-01		1.67E+02	5.44E+02	1.07E-01	
320,000	9.46E+01	7.39E+02	1.50E-01		9.59E+01	6.39E+02	1.37E-01		9.41E+01	3.11E+02	8.35E-02	
220,000	4.29E+01	3.49E+02	9.31E-02		4.30E+01	2.81E+02	7.72E-02		4.24E+01	1.40E+02	5.58E-02	
120,000	1.27E+01	7.26E+01	4.71E-02		1.23E+01	6.51E+01	3.74E-02		1.23E+01	3.92E+01	3.00E-02	
20,000	3.47E-01	9.73E-01	4.62E-03	3.57E+03	3.88E-01	1.00E+00	3.61E-02	3.57E+03	3.39E-01	9.45E-01	7.50E-03	3.57E+03

Figure 2.4: Average times on *kmonotone* sequences of `string`s, that are 32 letters long



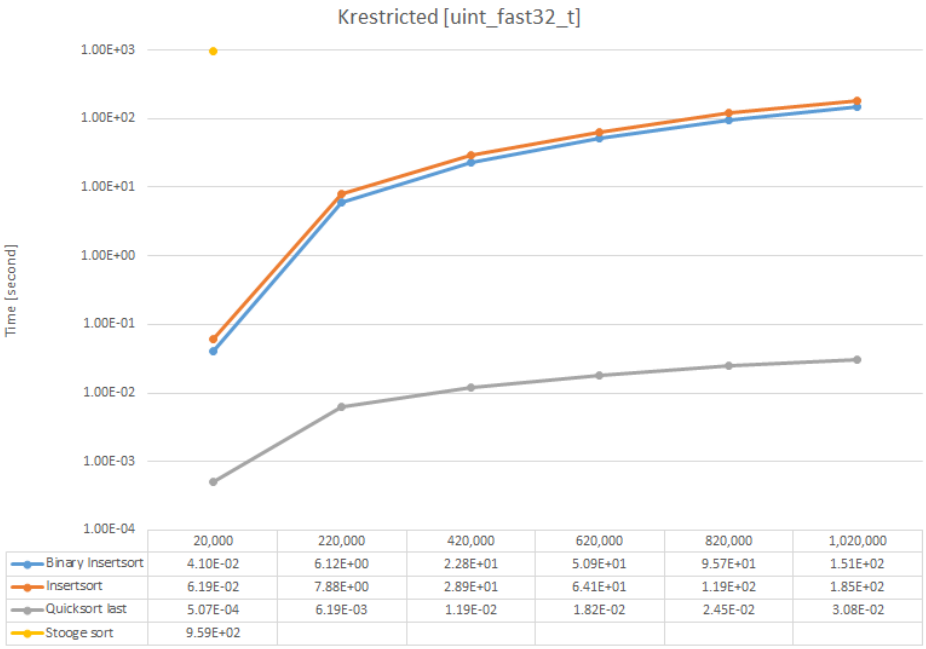
	16shuffled				32shuffled				64shuffled			
	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort
1,020,000	7.90E-02	1.30E-02	5.99E+01		8.03E-02	1.49E-02	3.40E+01		8.59E-02	2.03E-02	1.98E+01	
820,000	6.11E-02	9.91E-03	3.65E+01		6.61E-02	1.33E-02	2.05E+01		6.81E-02	1.62E-02	1.10E+01	
620,000	4.55E-02	7.68E-03	1.95E+01		4.80E-02	9.26E-03	1.18E+01		5.06E-02	1.23E-02	6.49E+00	
420,000	2.98E-02	4.90E-03	8.20E+00		3.16E-02	6.05E-03	4.93E+00		3.41E-02	8.33E-03	2.81E+00	
220,000	1.52E-02	2.78E-03	2.20E+00		1.63E-02	3.66E-03	1.34E+00		1.82E-02	4.85E-03	7.70E-01	
20,000	1.25E-03	2.33E-04	1.61E-02	9.66E+02	1.15E-03	2.90E-04	9.91E-03	9.59E+02	1.27E-03	3.96E-04	6.17E-03	9.65E+02

Figure 2.5: Average times on *kshuffled* sequences of `uint_fast32_t`

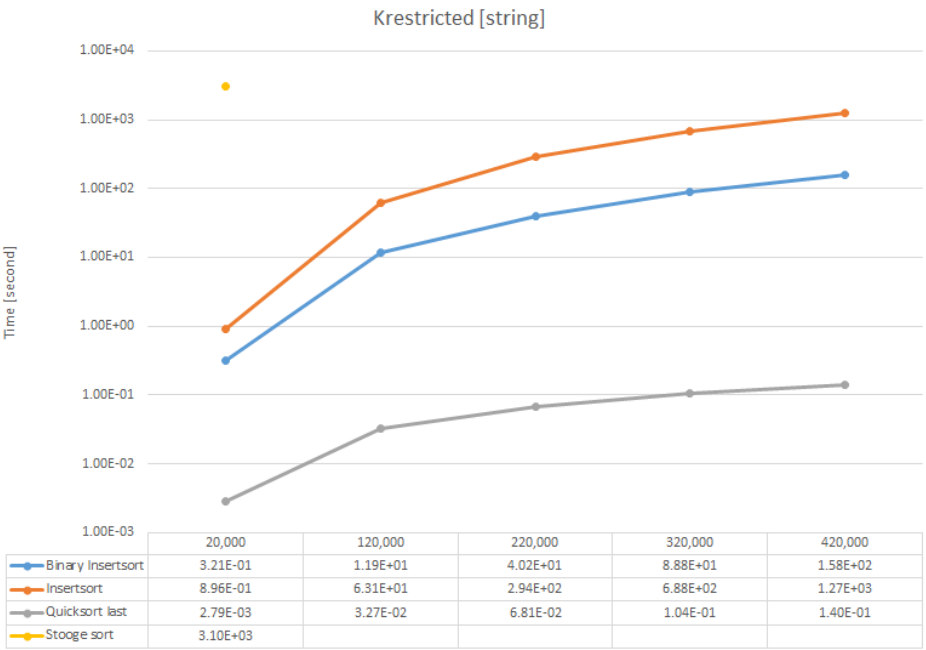


	16shuffled				32shuffled				64shuffled			
	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort
420,000	1.85E-01	6.59E-02	1.27E+02		2.05E-01	7.98E-02	1.01E+02		1.99E-01	1.10E-01	7.16E+01	
320,000	1.35E-01	4.94E-02	7.29E+01		1.41E-01	6.28E-02	5.80E+01		1.45E-01	8.38E-02	4.12E+01	
220,000	8.66E-02	3.38E-02	3.43E+01		8.31E-02	4.20E-02	2.77E+01		9.28E-02	5.80E-02	1.85E+01	
120,000	3.99E-02	1.84E-02	9.97E+00		4.28E-02	2.34E-02	7.06E+00		4.60E-02	3.15E-02	4.88E+00	
20,000	4.57E-03	3.13E-03	2.39E-01	3.30E+03	4.96E-03	4.33E-03	1.44E-01	3.34E+03	6.27E-03	5.35E-03	9.05E-02	3.54E+03

Figure 2.6: Average times on *kshuffled* sequences of `strings`, that are 32 letters long

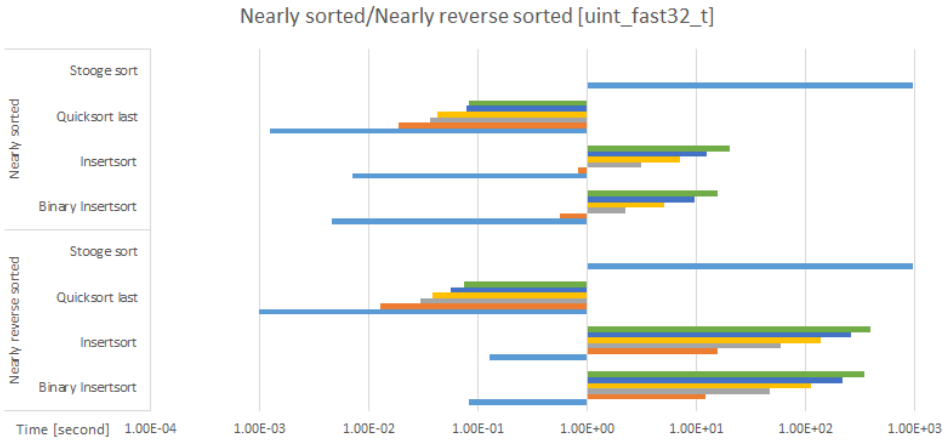


	20,000	220,000	420,000	620,000	820,000	1,020,000
Binary Insertsort	4.10E-02	6.12E+00	2.28E+01	5.09E+01	9.57E+01	1.51E+02
Insertsort	6.19E-02	7.88E+00	2.89E+01	6.41E+01	1.19E+02	1.85E+02
Quicksort last	5.07E-04	6.19E-03	1.19E-02	1.82E-02	2.45E-02	3.08E-02
Stooge sort	9.59E+02					

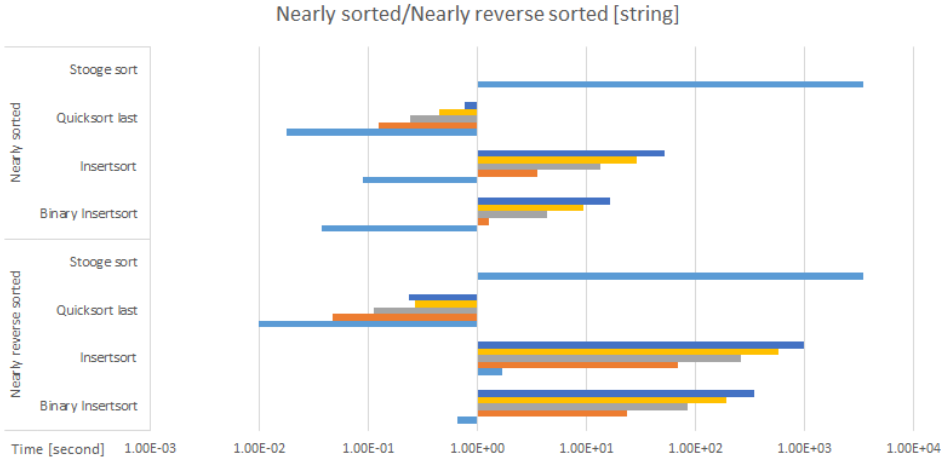


	20,000	120,000	220,000	320,000	420,000
Binary Insertsort	3.21E-01	1.19E+01	4.02E+01	8.88E+01	1.58E+02
Insertsort	8.96E-01	6.31E+01	2.94E+02	6.88E+02	1.27E+03
Quicksort last	2.79E-03	3.27E-02	6.81E-02	1.04E-01	1.40E-01
Stooge sort	3.10E+03				

Figure 2.7: Average times on *krestricted* sequences of `uint_fast32_t` and `strings`, that are 32 letters long

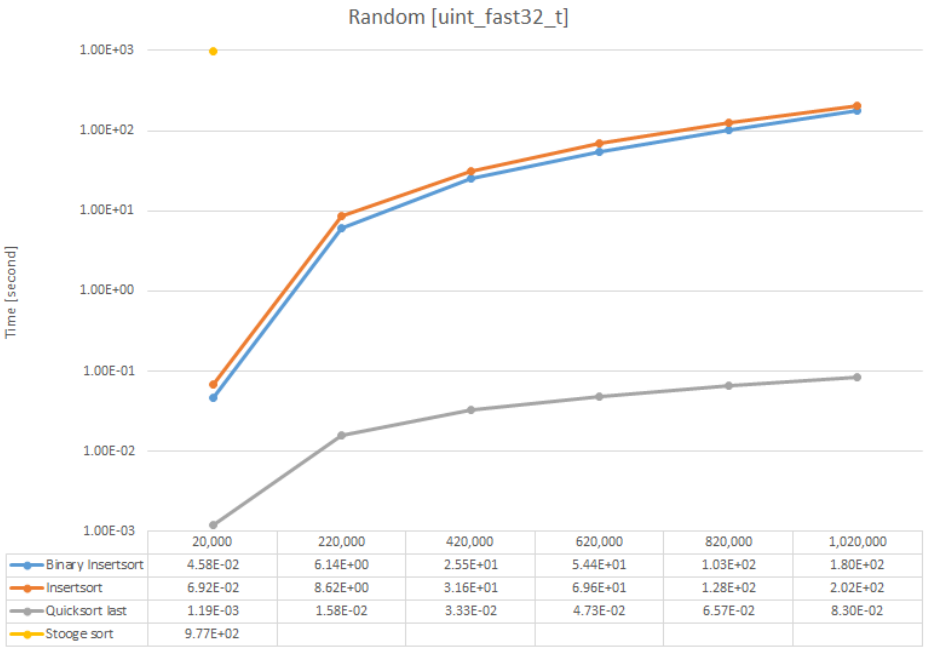


	Nearly reverse sorted				Nearly sorted			
	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort
1,020,000	3.50E+02	3.98E+02	7.44E-02		1.56E+01	2.03E+01	8.31E-02	
820,000	2.18E+02	2.65E+02	5.58E-02		9.59E+00	1.24E+01	7.82E-02	
620,000	1.14E+02	1.38E+02	3.81E-02		5.06E+00	7.11E+00	4.25E-02	
420,000	4.69E+01	5.91E+01	2.97E-02		2.25E+00	3.13E+00	3.68E-02	
220,000	1.21E+01	1.59E+01	1.29E-02		5.65E-01	8.40E-01	1.87E-02	
20,000	8.35E-02	1.27E-01	9.85E-04	9.65E+02	4.61E-03	7.09E-03	1.24E-03	9.66E+02

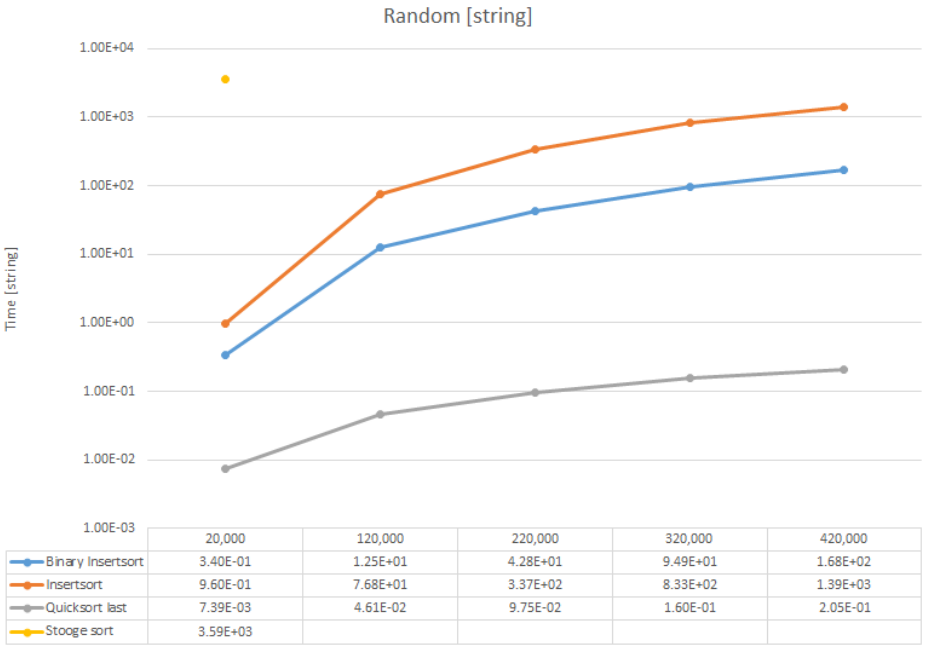


	Nearly reverse sorted				Nearly sorted			
	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort
420,000	3.40E+02	9.86E+02	2.36E-01		1.65E+01	5.19E+01	7.62E-01	
320,000	1.89E+02	5.67E+02	2.63E-01		9.31E+00	2.87E+01	4.38E-01	
220,000	8.43E+01	2.59E+02	1.12E-01		4.30E+00	1.33E+01	2.42E-01	
120,000	2.34E+01	6.81E+01	4.71E-02		1.27E+00	3.53E+00	1.22E-01	
20,000	6.56E-01	1.69E+00	9.75E-03	3.49E+03	3.73E-02	8.83E-02	1.75E-02	3.46E+03

Figure 2.8: Average times on *nearly sorted* and *nearly reverse sorted* sequences of `strings`, that are 32 letters long

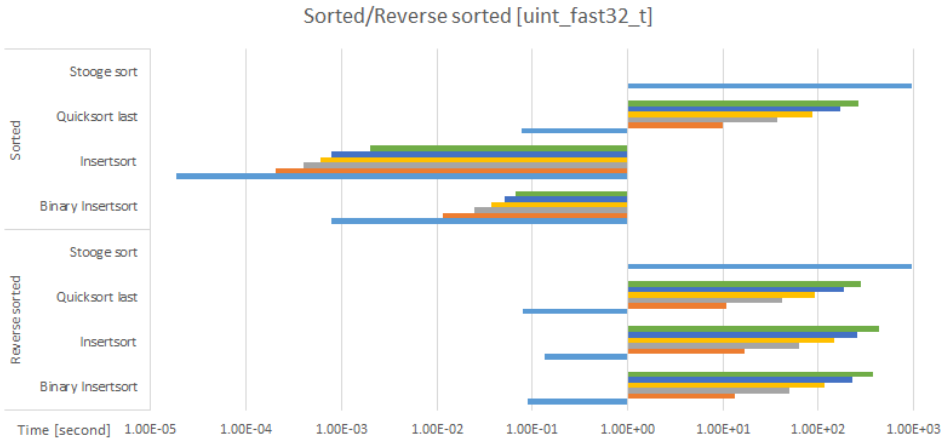


	20,000	220,000	420,000	620,000	820,000	1,020,000
Binary Insertsort	4.58E-02	6.14E+00	2.55E+01	5.44E+01	1.03E+02	1.80E+02
Insertsort	6.92E-02	8.62E+00	3.16E+01	6.96E+01	1.28E+02	2.02E+02
Quicksort last	1.19E-03	1.58E-02	3.33E-02	4.73E-02	6.57E-02	8.30E-02
Stooge sort	9.77E+02					

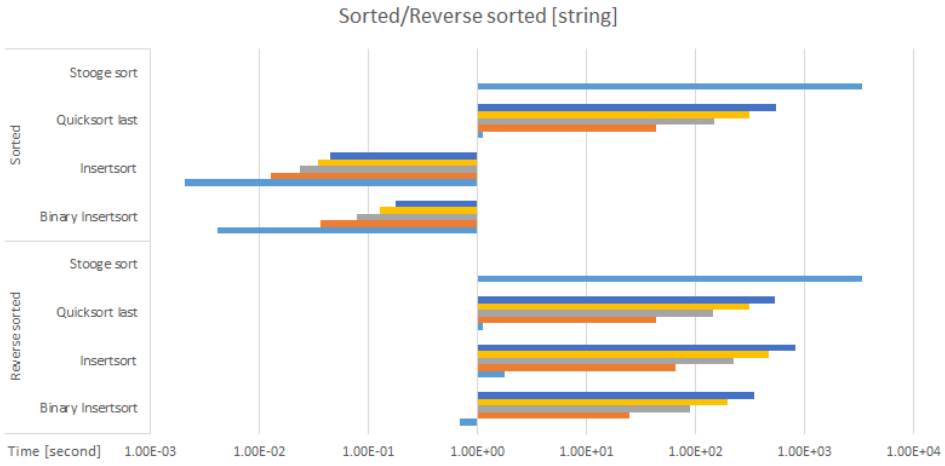


	20,000	120,000	220,000	320,000	420,000
Binary Insertsort	3.40E-01	1.25E+01	4.28E+01	9.49E+01	1.68E+02
Insertsort	9.60E-01	7.68E+01	3.37E+02	8.33E+02	1.39E+03
Quicksort last	7.39E-03	4.61E-02	9.75E-02	1.60E-01	2.05E-01
Stooge sort	3.59E+03				

Figure 2.9: Average times on *random* sequences of `uint_fast32_t` and `strings`, that are 32 letters long



	Reverse sorted				Sorted			
	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort
1,020,000	3.78E+02	4.27E+02	2.82E+02		6.60E-02	1.98E-03	2.64E+02	
820,000	2.30E+02	2.56E+02	1.83E+02		5.16E-02	7.83E-04	1.71E+02	
620,000	1.15E+02	1.45E+02	9.30E+01		3.72E-02	6.00E-04	8.63E+01	
420,000	5.01E+01	6.35E+01	4.13E+01		2.48E-02	3.96E-04	3.73E+01	
220,000	1.32E+01	1.68E+01	1.09E+01		1.17E-02	2.05E-04	1.01E+01	
20,000	8.84E-02	1.35E-01	8.07E-02	9.64E+02	7.75E-04	1.86E-05	7.75E-02	9.66E+02



	Reverse sorted				Sorted			
	Binary Insertsort	Insertsort	Quicksort last	Stooge sort	Binary Insertsort	Insertsort	Quicksort last	Stooge sort
420,000	3.46E+02	8.31E+02	5.38E+02		1.79E-01	4.43E-02	5.41E+02	
320,000	1.97E+02	4.73E+02	3.14E+02		1.26E-01	3.46E-02	3.14E+02	
220,000	8.82E+01	2.24E+02	1.45E+02		7.77E-02	2.32E-02	1.50E+02	
120,000	2.46E+01	6.60E+01	4.29E+01		3.64E-02	1.28E-02	4.35E+01	
20,000	6.81E-01	1.78E+00	1.12E+00	3.41E+03	4.15E-03	2.08E-03	1.12E+00	3.39E+03

Figure 2.10: Average times on *sorted* and *reverse sorted* sequences of `uint_fast32_t` and `strings`, that are 32 letters long

2.5.3 The third part of measurings

In the third part, we have tested the heapsort, mergesort, timsort and quicksort variants, that used random and median of three pivots. All of them were tested 2 times for each size, sequence and element type. For the `uint_fast32_t` element type, the tested sizes were multiples of $100,000,000$, from $100,000,000$ up to $500,000,000$. However, these weren't feasible for `string`s of length 32, due to excessive memory requirements. Instead, they were tested on multiples of $10,000,000$, from $10,000,000$ up to $50,000,000$.

K-monotone

The *kmonotone* sequences were tested for K equal to 2, 4 and 32. The sequences with K equal to 2 are nearly identical to the random ones, except that there are certainly no longer sorted or reverse sorted subsequences, that are likely to occur in extremely long random input.

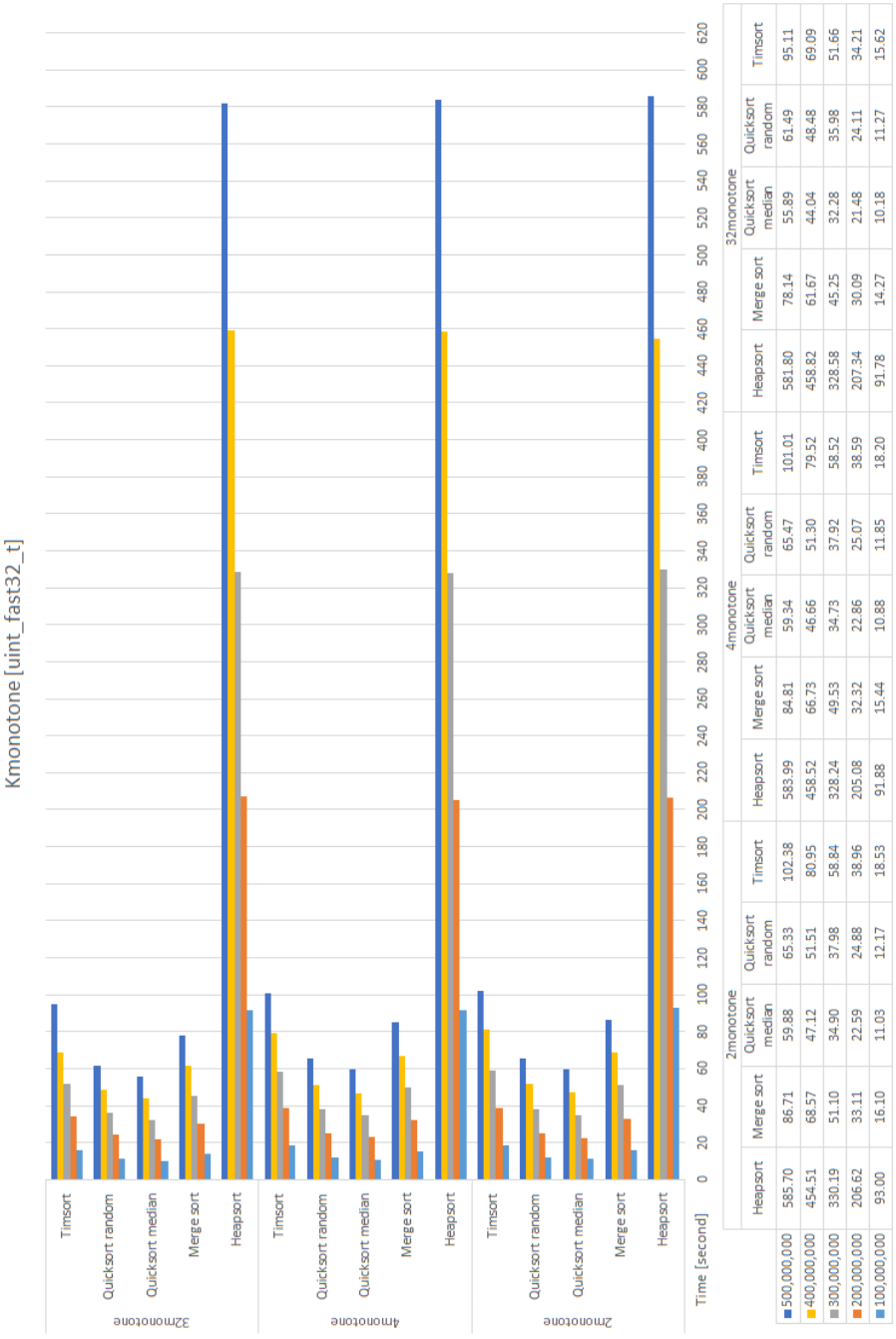
The *2monotone* sequence type in combination with `uint_fast32_t` and `string` element type triggered for heapsort the worst case times. The measured times were on par with those of the *random* sequences. However, with the growing K, the measured times have become slightly better. What's even more interesting is, that this phenomenon manifested only for `string` element type. In further investigation, we have measured amount of comparisons and element moves. The measurements

Sequence type	`string`/$30,000,000$	
	Comparisons	Element moves
2monotone	$715,131,825$	$868,370,944$
	$715,134,909$	$868,377,302$
4monotone	$717,325,401$	$866,230,576$
	$717,328,678$	$866,207,944$
32monotone	$722,678,524$	$861,303,497$
	$722,698,768$	$861,289,442$

showed, that the number of comparisons grew at the same pace as the number of element moves lowered. The likely cause for this is, that amount of randomness lowered, what triggered during heap building more comparisons in Bottom-up-search and conversely less element moves in interchange subroutine.

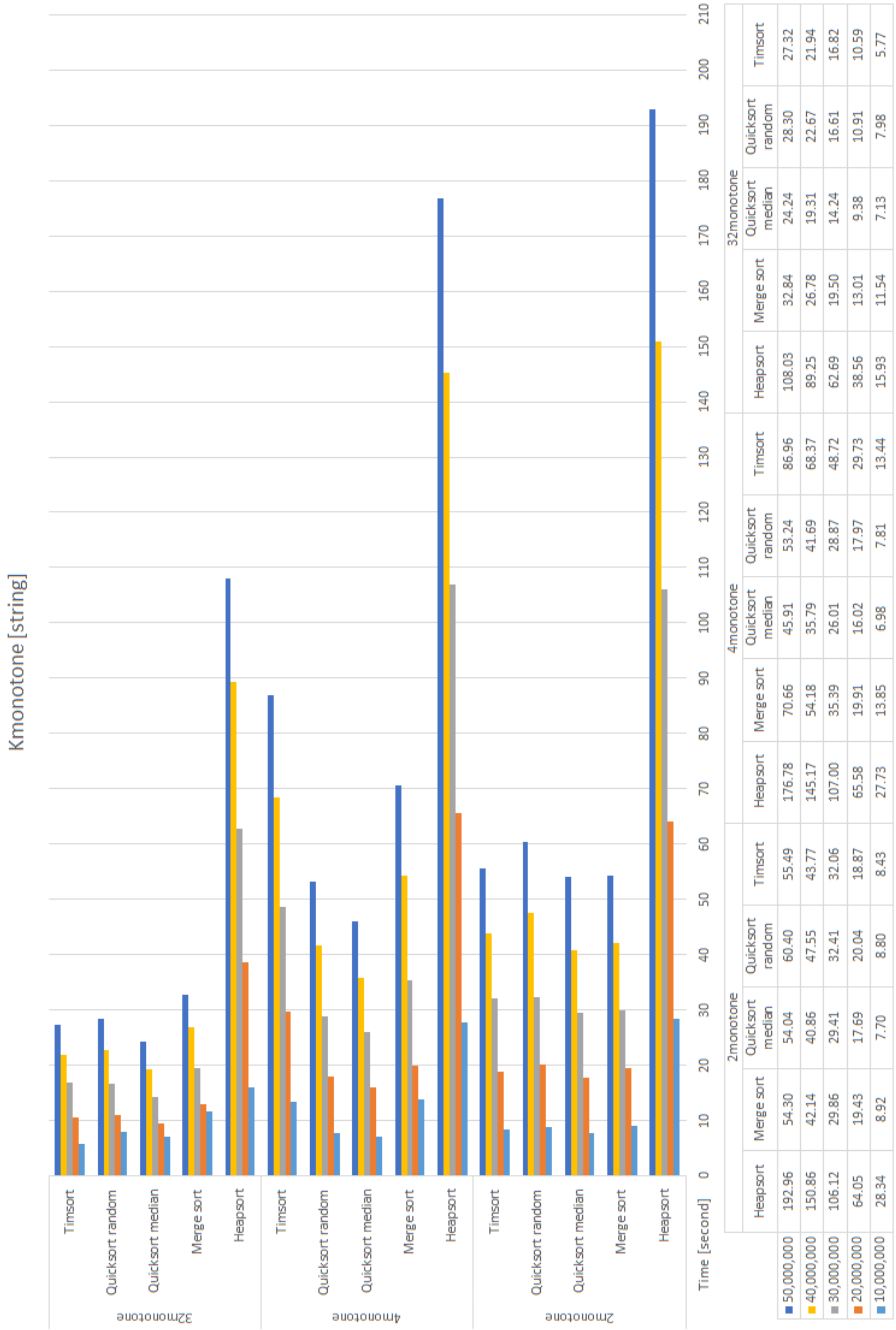
Next, we have found out, that timsort is not necessarily faster than mergesort. For `uint_fast32_t` element type and K equal to 2, 4 and 32, it was noticeably slower. To confirm this result on larger K, we have done additional tests. These showed, that for K larger than 150, timsort is faster than mergesort on $100,000,000$ elements and that for K equal to 32, it is faster, when input is at most $81,500$ long. Thus, timsort is superior to mergesort on very short sequences of `uint_fast32_t` and on all longer sequences of `uint_fast32_t`, that have improbably long monotone subsequences. On the other hand, when we switched to `string`s of length 32, timsort performed better for K equal to 32. There are at least two likely reasons, that substantiate such threshold. Firstly, binary insertion sort for the K close to run length performed less swaps than mergesort. Secondly, timsort carried out a smaller amount of element moves, because it copied each time the smaller run to the temporary array. Thirdly, since, the *kmonotone* sequences are constructed randomly, the effectivity of preliminary searches and galloping is very limited. However, for sequences in tens of millions, it should have impact, albeit possibly a small one.

In addition, we can conclude for *kmonotone* sequences, that the quicksort's partitioning approach is faster than the merging one. On `uint_fast32_t` sequences with K equal to 2, 4 and 32, timsort and mergesort were quite a bit slower than both quicksorts. The only exception to this were sequences of `string`s with K greater or equal to 32, where timsort achieved better times than the quicksort with random pivot. From the practical point of view, the K required for it to be faster is very unlikely. Because of that, quicksorts are a better choice for *kmonotone* sequences.



	2monotone					4monotone					32monotone				
	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort
500,000,000	585.70	86.71	59.88	65.33	102.38	583.99	84.81	59.34	65.47	101.01	581.80	78.14	55.89	61.49	95.11
400,000,000	454.51	68.57	47.12	51.51	80.95	458.52	66.73	46.66	51.30	79.52	458.82	61.67	44.04	48.48	69.09
300,000,000	330.19	51.10	34.90	37.98	58.84	328.24	49.53	34.73	37.92	58.52	328.58	45.25	32.28	35.98	51.66
200,000,000	206.62	33.11	22.59	24.88	38.96	205.08	32.32	22.86	25.07	38.59	207.34	30.09	21.48	24.11	34.21
100,000,000	93.00	16.10	11.03	12.17	18.53	91.88	15.44	10.88	11.85	18.20	91.78	14.27	10.18	11.27	15.62

Figure 2.11: Average times on *kmonotone* sequences of `uint_fast32_t`



	2monotone					4monotone					32monotone				
	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort
50,000,000	192.96	54.30	54.04	60.40	55.49	176.78	70.66	45.91	53.24	86.96	108.03	32.84	24.24	28.30	27.32
40,000,000	150.86	42.14	40.86	47.55	43.77	145.17	54.18	35.79	41.69	68.37	89.25	26.78	19.31	22.67	21.94
30,000,000	106.12	29.86	29.41	32.41	32.06	107.00	35.39	26.01	28.87	48.72	62.69	19.50	14.24	16.61	16.82
20,000,000	64.05	19.43	17.69	20.04	18.87	65.58	19.91	16.02	17.97	29.73	38.56	13.01	9.38	10.91	10.59
10,000,000	28.34	8.92	7.70	8.80	8.43	27.73	13.85	6.98	7.81	13.44	15.93	11.54	7.13	7.98	5.77

Figure 2.12: Average times on *kmonotone* sequences of `string`s, that are 32 letters long

K-shuffled

On the contrary to the *kmonotone* sequence type, the best performing sorting algorithm for *kshuffled* sequences was by far timsort. Its advantage over other sorting algorithms stems predominantly from the construction of *kshuffled* sequences, that are created by randomly shuffling each successive K elements of some sorted sequence. The major costs for timsort are then caused by the initial binary insertion sorting. Following initial sorting, there are no inversions between runs on the stack and merging is suspended, because the preliminary search finds out in logarithmic number of steps, that each two consecutive runs already form a sorted run.

The second best performer was quicksort with median of three pivot. Compared to random pivot quicksort, that ended up third, it performed around 27.83% better on `uint_fast32_t` and 33.06% better on `string`s. Its superiority at large stems from the choice of elements used to compute a median. For some K, there's $1/K$ probability, that the middle element will be at the same time a median of the whole sequence. However, as the K grows, this advantage diminishes and it should perform on par, or little bit better than random pivot quicksort due to better average number of comparisons.

The most counterintuitive result obtained by testing is, that mergesort is noticeably slower on shuffled sequences. To understand what has happened, we have derived a number of comparisons and element moves for some K and length n. The corner stone here is that after initial $\log(K)$ levels of merges, the number of required element moves and comparisons lowered by a third and half, respectively. All in all, it made approximately

$$\frac{3n}{2}\log_2 K + n(\lceil\log_2 n\rceil - \log_2 K)$$

element moves and

$$n\log_2 K + \frac{n}{2}(\lceil\log_2 n\rceil - \log_2 K)$$

comparisons. When compared to the quicksort's average number of moves ($0.6931n\log n$ [11, p. 294]) and comparisons ($1.39n\log n$ [11, p. 294]), mergesort should have advantage only in number of comparisons. Additional measurements, as presented in the following table confirmed this assumption. For both element types, mergesort made half the

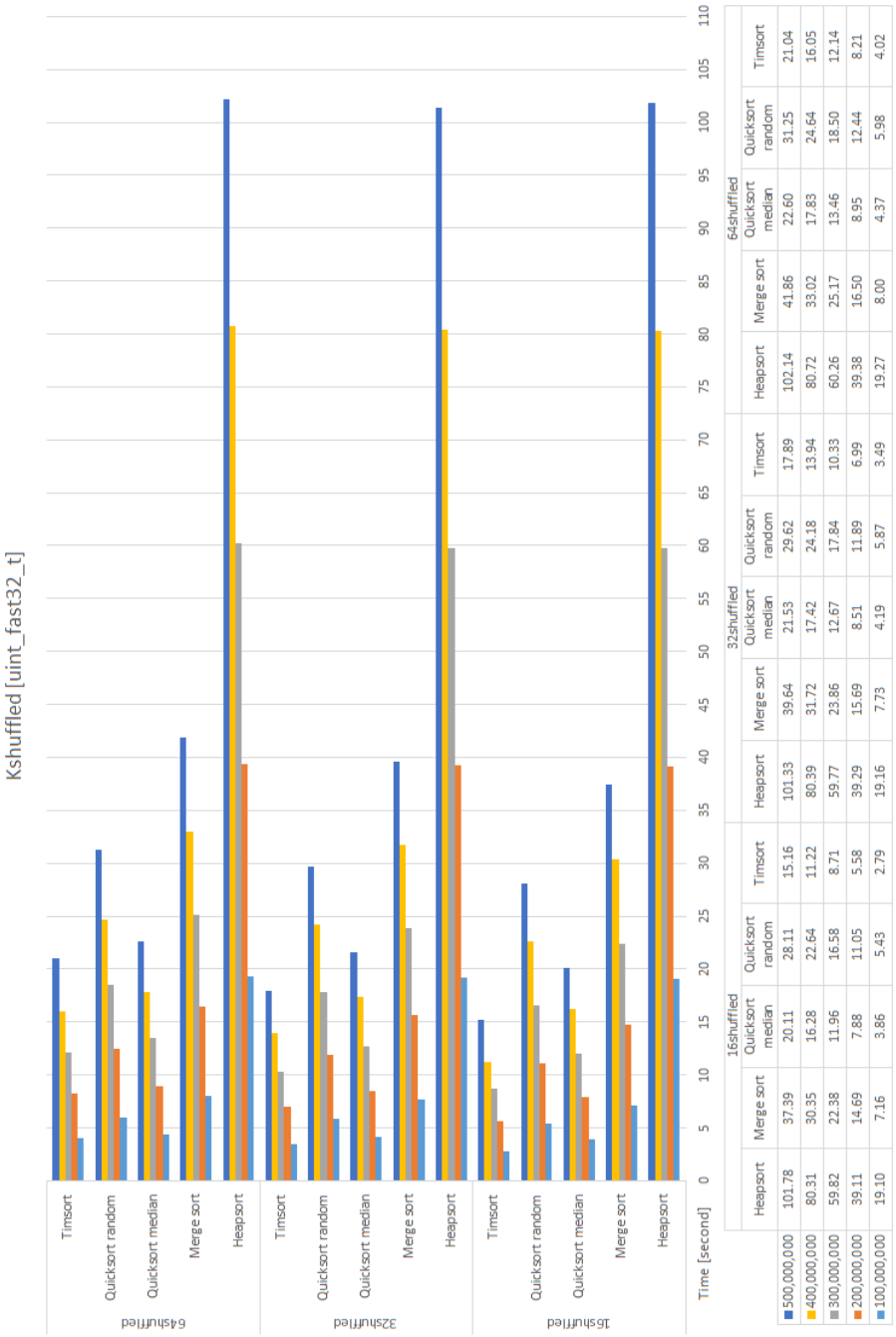
number of comparisons quicksort with random element pivot did and more than four times more element moves. The interesting change appeared on `string`s, where the time difference was only the minor one. Even though the ratio of executed operations remained the same, the higher costs of single comparison had a great impact.

	16shuffled sequence type			
Algorithms	`string`/30,000,000		`uint_fast32_t`/300,000,000	
	Comparisons	Element moves	Comparisons	Element moves
Mergesort	422,636,731	802,438,779	4,853,204,923	9,278,080,699
	422,632,385	802,434,433	4,853,201,098	9,278,076,874
Quicksort random pivot	974,620,295	165,828,357	10,376,838,118	1,658,255,436
	943,403,017	165,827,889	10,545,819,623	1,658,293,566

	32shuffled sequence type			
Algorithms	`string`/30,000,000		`uint_fast32_t`/300,000,000	
	Comparisons	Element moves	Comparisons	Element moves
Mergesort	436,757,281	816,559,329	4,994,383,383	9,419,259,159
	436,753,494	816,555,542	4,994,405,215	9,419,280,991
Quicksort random pivot	905,083,995	181,099,797	10,567,887,435	1,810,966,995
	916,982,271	181,084,269	10,623,275,645	1,810,969,131

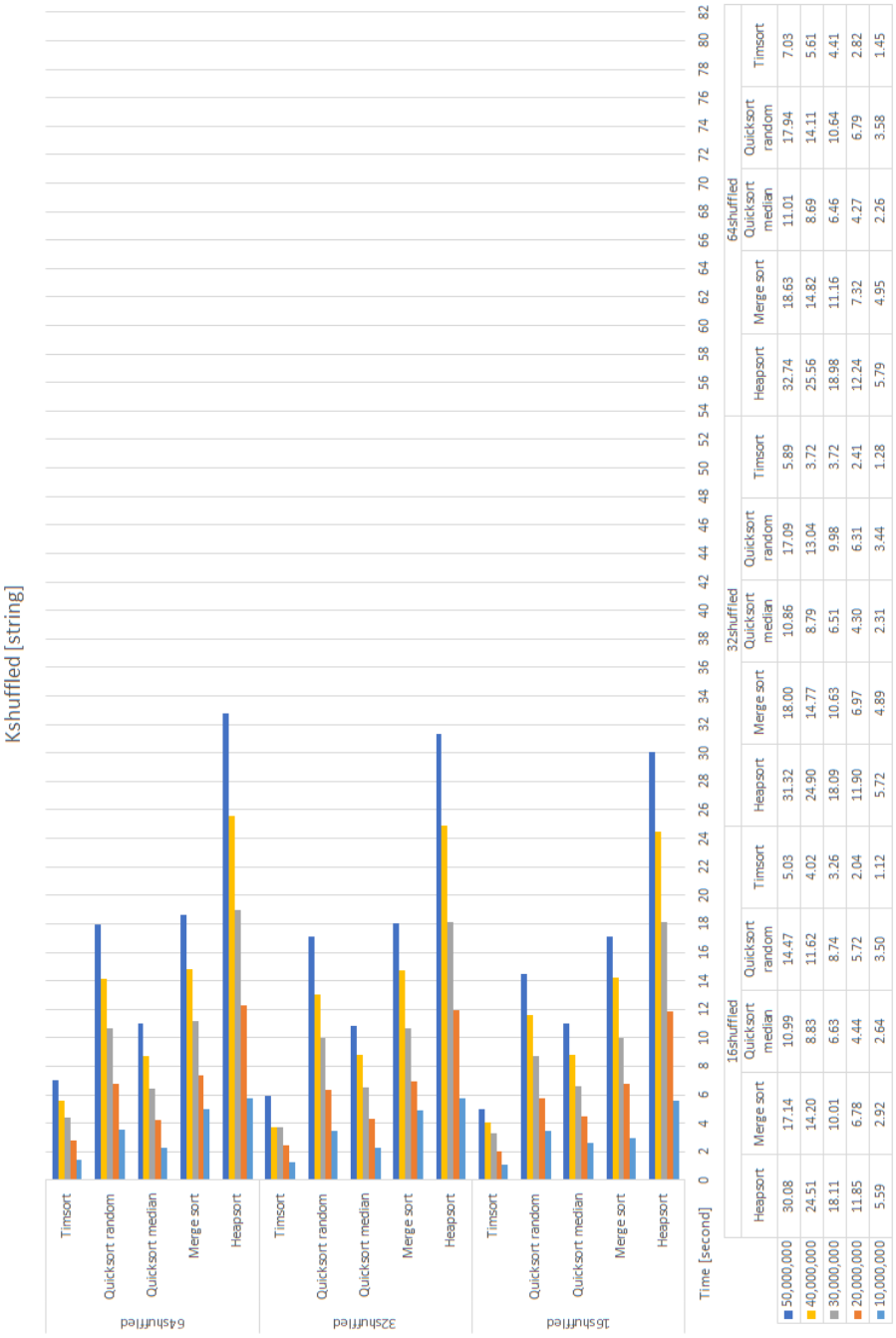
	64shuffled sequence type			
Algorithms	`string`/30,000,000		`uint_fast32_t`/300,000,000	
	Comparisons	Element moves	Comparisons	Element moves
Mergesort	451,295,035	831,097,083	5,139,830,126	9,564,705,902
	451,301,867	831,103,915	5,139,850,054	9,564,725,830
Quicksort random pivot	880,570,590	198,177,792	10,374,716,471	1,981,723,716
	885,567,882	198,183,894	10,173,144,171	1,981,723,995

Lastly, the worst performing sorting algorithm was heapsort. Its dependency on custom data structure and very low locality of reference took its price.



	16shuffled					32shuffled					64shuffled				
	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort
500,000,000	101.78	37.39	20.11	28.11	15.16	101.33	39.64	21.53	29.62	17.89	102.14	41.86	22.60	31.25	21.04
400,000,000	80.31	30.35	16.28	22.64	11.22	80.39	31.72	17.42	24.18	13.94	80.72	33.02	17.83	24.64	16.05
300,000,000	59.82	22.38	11.96	16.58	8.71	59.77	23.86	12.67	17.84	10.33	60.26	25.17	13.46	18.50	12.14
200,000,000	39.11	14.69	7.88	11.05	5.58	39.29	15.69	8.51	11.89	6.99	39.38	16.50	8.95	12.44	8.21
100,000,000	19.10	7.16	3.86	5.43	2.79	19.16	7.73	4.19	5.87	3.49	19.27	8.00	4.37	5.98	4.02

Figure 2.13: Average times on *kshuffled* sequences of `uint_fast32_t`

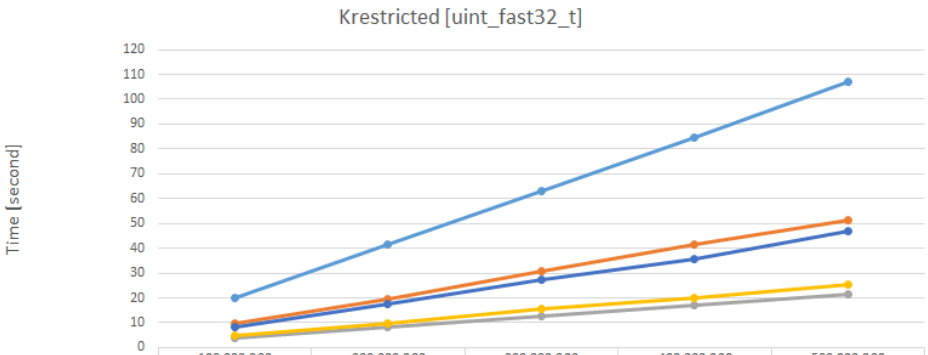


	16shuffled					32shuffled					64shuffled				
	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort
50,000,000	30.08	17.14	10.99	14.47	5.03	31.32	18.00	10.86	17.09	5.89	32.74	18.63	11.01	17.94	7.03
40,000,000	24.51	14.20	8.83	11.62	4.02	24.90	14.77	8.79	13.04	3.72	25.56	14.82	8.69	14.11	5.61
30,000,000	18.11	10.01	6.63	8.74	3.26	18.09	10.63	6.51	9.98	3.72	18.98	11.16	6.46	10.64	4.41
20,000,000	11.85	6.78	4.44	5.72	2.04	11.90	6.97	4.30	6.31	2.41	12.24	7.32	4.27	6.79	2.82
10,000,000	5.59	2.92	2.64	3.50	1.12	5.72	4.89	2.31	3.44	1.28	5.79	4.95	2.26	3.58	1.45

Figure 2.14: Average times on *kshuffled* sequences of `strings`, that are 32 letters long

K-restricted

On *krestricted* sequences, there wasn't a single best performer. Similarly to the *kmonotone* sequences, timsort outperformed both quicksorts on the `string` element type. The most reasonable explanation for this result would be, that timsort carried out smaller number of costly comparisons thanks to the galloping and preliminary search techniques. On the other hand, quicksorts with random and median of three pivot were nearly twice as fast on `uint_fast32_t` element type. A big difference between measured times was probably caused by timsort's high merging costs, that predominated over low `uint_fast32_t` comparison costs. Even the results of mergesort support this theory, because unlike timsort, it performed quite bad on sequences of `string`s.

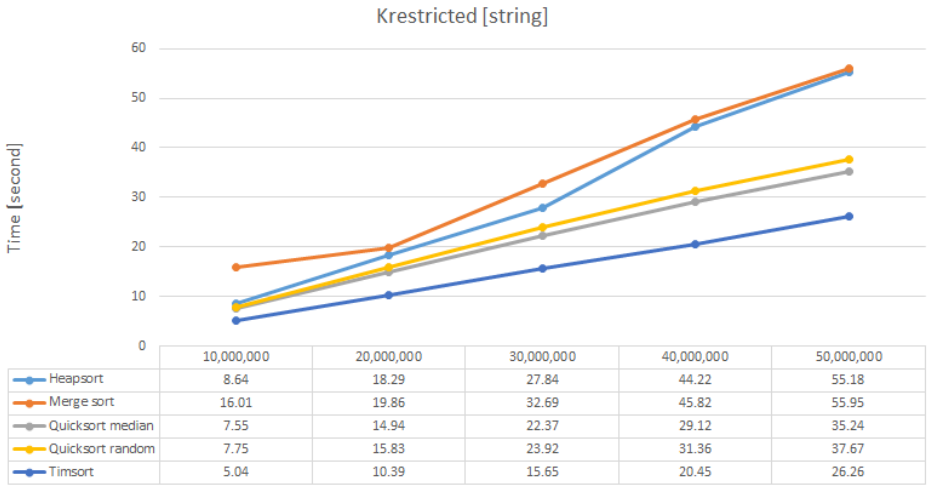


	100,000,000	200,000,000	300,000,000	400,000,000	500,000,000
Heapsort	20.04	41.30	62.81	84.76	107.28
Merge sort	9.63	19.68	30.94	41.35	51.21
Quicksort median	3.96	8.25	12.83	17.18	21.44
Quicksort random	4.67	9.85	15.41	20.17	25.27
Timsort	8.50	17.55	27.31	35.91	46.67

Figure 2.15: Average times on *krestricted* sequences of `uint_fast32_t`

The worst performing sorting algorithm differed for used element type. For `uint_fast32_t`, the worst one was heapsort. Compared to mergesort, it was twice as slow. On the contrary, for `string`s of length 32, heapsort performed on par with mergesort or slightly better. To examine this phenomenon, we have done additional tests, that measured number of comparisons and element moves.

On $30,000,000$ long sequences of `string`s, mergesort carried out around 14.5 millions less comparisons and 271 millions more element moves. This measurement would correspond to the fact, that in each



	10,000,000	20,000,000	30,000,000	40,000,000	50,000,000
Heapsort	8.64	18.29	27.84	44.22	55.18
Merge sort	16.01	19.86	32.69	45.82	55.95
Quicksort median	7.55	14.94	22.37	29.12	35.24
Quicksort random	7.75	15.83	23.92	31.36	37.67
Timsort	5.04	10.39	15.65	20.45	26.26

Figure 2.16: Average times on *krestricted* sequences of `strings`, that are 32 letters long

Algorithms	`string`/30,000,000		`uint_fast32_t`/300,000,000	
	Comparisons	Element moves	Comparisons	Element moves
Heapsort	746,184,341	839,171,399	8,450,051,101	9,392,593,682
	746,152,537	839,206,887	8,449,991,971	9,392,661,796
	746,137,825	839,213,387	8,450,055,544	9,392,608,590
	746,161,447	839,202,170	8,450,035,731	9,392,633,263
Mergesort	731,677,734	1,111,479,782	8,434,275,408	12,859,151,184
	731,664,777	1,111,466,825	8,434,307,263	12,859,183,039
	731,672,681	1,111,474,729	8,434,297,232	12,859,173,008
	731,683,045	1,111,485,093	8,434,289,174	12,859,164,950

merge, it made $3n/2$ element moves and $\frac{3n}{2} \log n$ together, as compared to $n \log n$ average case of heapsort. Overall, mergesort's advantage in the number of comparisons was nearly able to balance additional element moves, even though the caching effectiveness was limited by a larger size of strings.

On 300,000,000 long sequences of `uint_fast32_t`s, mergesort made around 15.7 millions less comparisons and 3.5 billions more element moves. Despite the staggering amount of element moves, it still performed two times better. This result was most probably caused by two factors. First, the *krestricted* sequences have a very few unique elements, that create a very long sorted subsequences in upper levels of merging.

Combined with the proper caching and prefetching, mergesort was able to perform more efficiently.

Nearly sorted and reverse sorted

On nearly sorted sequences, that have at most 8% of displaced elements, the best performer for both element types was quicksort with median of three pivot. With the very few displaced elements, it chose each time a favourable pivot, what resulted in less partitioning rounds.

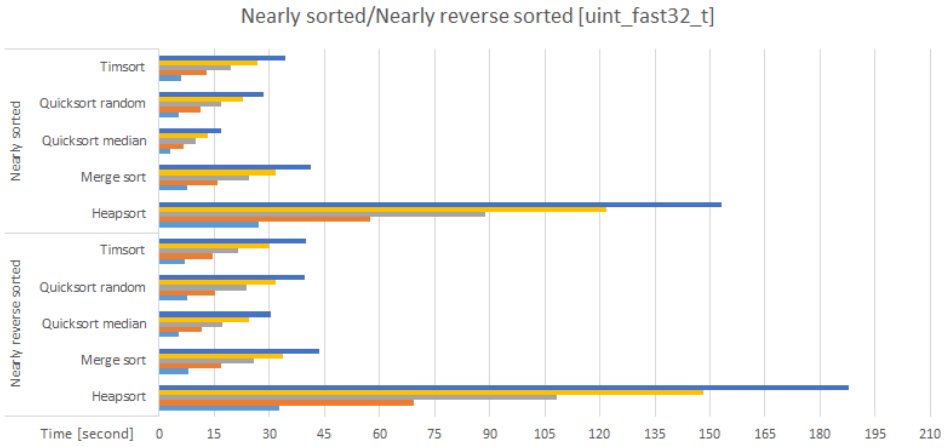
Furthermore, the quicksort with random pivot performed approximately same as timsort. Based on measured times, we can tell, that both timsort and quicksort had additional workload on nearly reverse sorted sequences, when compared to nearly sorted ones. The timsort had to reverse discovered runs and quicksort had to swap more elements, when partitioning. This resulted in timsort being slightly faster on nearly reverse sorted sequences. On the contrary, the quicksort was faster on nearly sorted sequences.

The fourth best performer was mergesort. It fell behind timsort and quicksorts, because it did a lot more element moves and comparisons. On nearly reverse sorted sequences, it made about a third more element moves than on nearly sorted sequences. In addition, it carried out a lot of superfluous comparisons, given that the runs being merged were sorted for the most part. In all such cases, timsort performed more optimally.

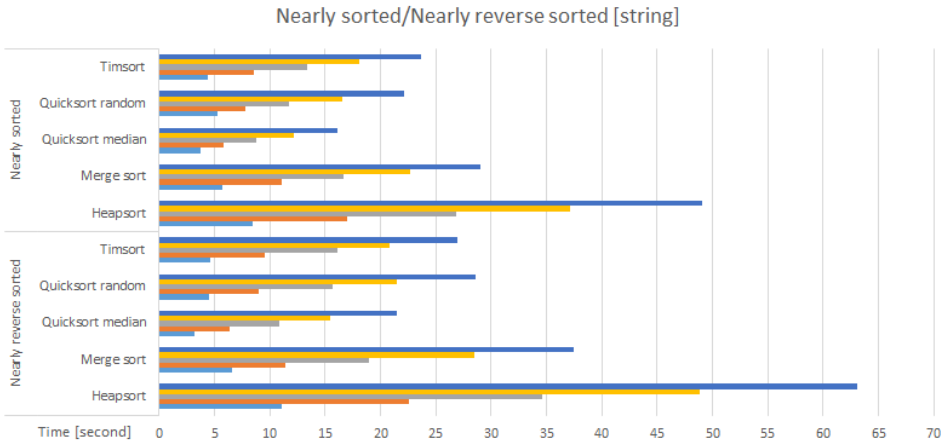
The worst sorting algorithm for this sequence type was again heapsort, that was as much as three times slower on `string`s and six times on `uint_fast32_t`. Based on measured times, we can also conclude, that the heap building costs were much higher for nearly reverse sorted sequences.

Random

On *random* sequences, the relative order of sorting algorithms didn't change very much. The only difference was, that mergesort was a little bit faster than timsort. Seemingly, its preliminary searches and galloping didn't pay off at all and took its price. In overall, the fastest sorting al-



	Nearly reverse sorted					Nearly sorted				
	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort
500,000,000	187.95	43.44	30.29	39.72	39.79	153.17	41.33	16.70	28.40	34.48
400,000,000	148.29	33.70	24.55	31.60	30.01	121.89	31.82	13.14	22.78	26.59
300,000,000	108.11	25.81	17.04	23.66	21.31	88.79	24.54	9.87	16.84	19.50
200,000,000	69.35	16.97	11.61	15.32	14.39	57.46	15.89	6.50	11.10	12.83
100,000,000	32.67	7.96	5.17	7.44	6.91	27.17	7.60	3.14	5.45	5.99

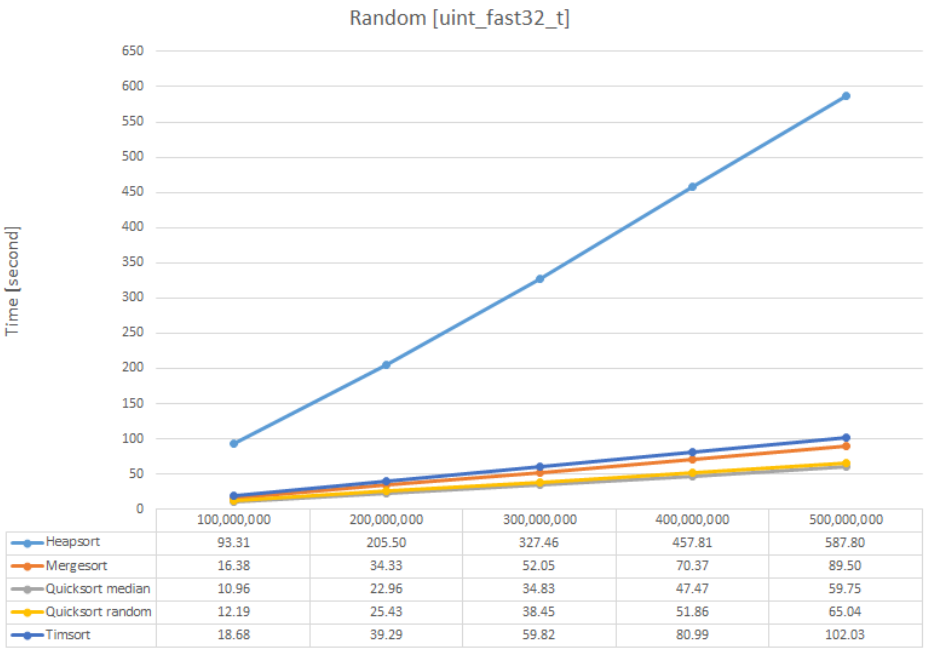


	Nearly reverse sorted					Nearly sorted				
	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort
50,000,000	63.09	37.44	21.46	28.56	26.94	49.03	28.97	16.06	22.12	23.66
40,000,000	48.84	28.48	15.43	21.45	20.86	37.18	22.63	12.14	16.55	18.05
30,000,000	34.56	18.96	10.85	15.70	16.09	26.84	16.64	8.71	11.76	13.37
20,000,000	22.57	11.34	6.31	9.01	9.50	17.02	11.03	5.84	7.74	8.56
10,000,000	11.02	6.61	3.16	4.51	4.55	8.43	5.67	3.68	5.29	4.43

Figure 2.17: Average times on *nearly sorted* and *nearly reverse sorted* sequences of `uint_fast32_t` and `strings`, that are 32 letters long

gorithm for random sequences was quicksort with median of three pivot.

The worst sorting algorithm was heapsort, that was as much as ten times slower.

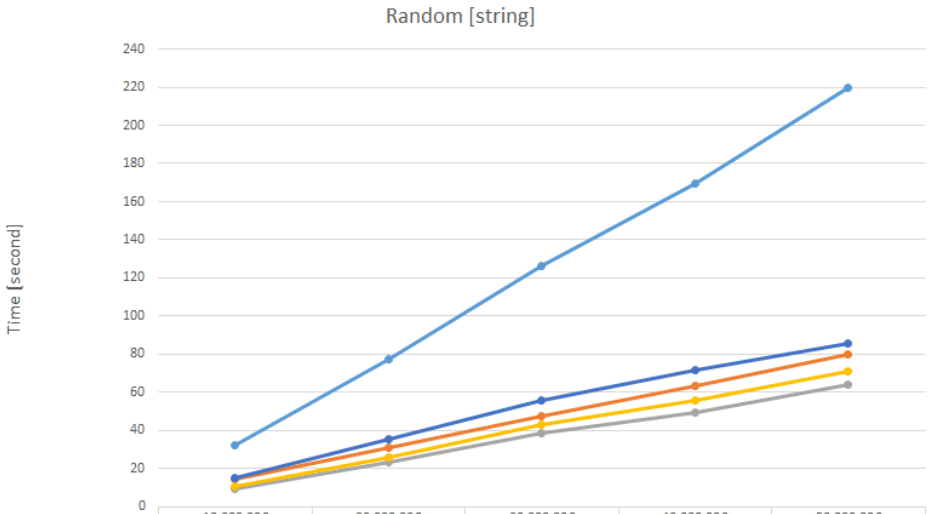


	100,000,000	200,000,000	300,000,000	400,000,000	500,000,000
Heapsort	93.31	205.50	327.46	457.81	587.80
Mergesort	16.38	34.33	52.05	70.37	89.50
Quicksort median	10.96	22.96	34.83	47.47	59.75
Quicksort random	12.19	25.43	38.45	51.86	65.04
Timsort	18.68	39.29	59.82	80.99	102.03

Figure 2.18: Average times on *random* sequences of `uint_fast32_t`

Sorted and reverse sorted

On sorted sequences, the best performing sorting algorithm was without a doubt timsort. Due to its adaptiveness, it was able to prevent merging costs and terminate in linear time. On the other hand, the best algorithms for reverse sorted sequences were quicksorts. For the `uint_fast32_t` element type, the median pivot choice was about 32.16% more effective than the random one. However, on `string`s of length 32, the tides has turned and quicksort with random pivot was faster by 12.29%. This change corresponded to a number of comparisons and element moves. Compared to median pivot quicksort, it performed about third less of both.



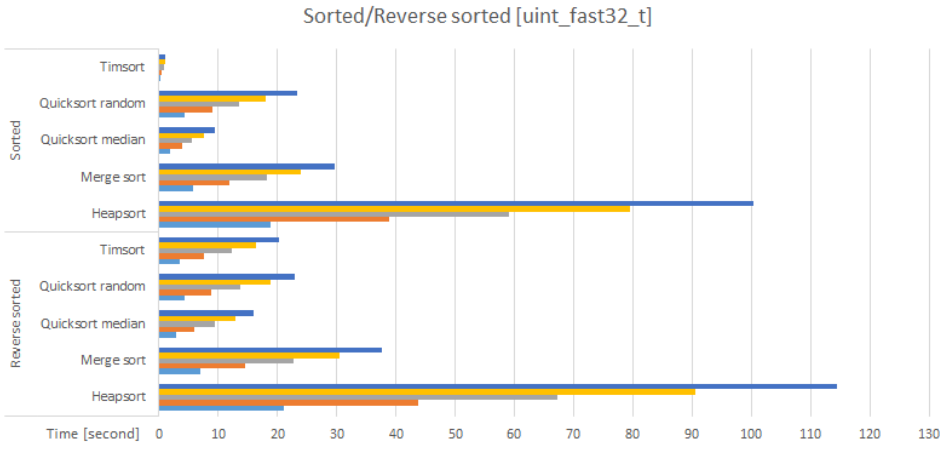
	10,000,000	20,000,000	30,000,000	40,000,000	50,000,000
Heapsort	32.05	77.09	126.22	169.42	219.40
Merge sort	14.23	30.87	47.33	63.50	80.15
Quicksort median	9.71	23.54	38.51	49.35	64.35
Quicksort random	10.98	25.73	43.10	55.81	70.84
Timsort	15.10	35.23	55.56	71.71	85.51

Figure 2.19: Average times on *random* sequences of `strings`, that are 32 letters long

2.5.4 Improvements using binary insertion sort

In the last part of the performance analysis, we have done additional measurements on modified mergesort and quicksorts with random, last and median of three pivots to see the influence of binary insertion sort on running times. In doing so, we have also experimentally determined the best possible bound, that triggers binary insertion sort to be 16. The major time difference was recorded for mergesort, which reached more than ten seconds better times for the largest tested input instances. As a result, it was more competetive with random pivot quicksort. It was most evident on *nearly reverse sorted* sequences of `uint_fast32_t`, where mergesort was slightly faster. Furthermore, when compared to timsort, mergesort with binary insertsort was faster on *krestricted* sequences of `uint_fast32_t` and *32monotone* sequences of both types.

From among quicksorts, the largest improvement could be seen for random pivot quicksort, although, it wasn't able to outperform quicksort



	Reverse sorted					Sorted				
	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort
500,000,000	114.48	37.56	15.89	22.82	20.32	100.25	29.70	9.49	23.32	1.14
400,000,000	90.47	30.46	12.80	18.90	16.45	79.55	23.90	7.65	18.08	1.01
300,000,000	67.31	22.69	9.50	13.72	12.31	59.03	18.20	5.54	13.58	0.75
200,000,000	43.69	14.45	5.97	8.85	7.57	38.82	11.97	3.82	9.11	0.52
100,000,000	21.15	7.00	2.84	4.36	3.52	18.92	5.79	1.84	4.42	0.26

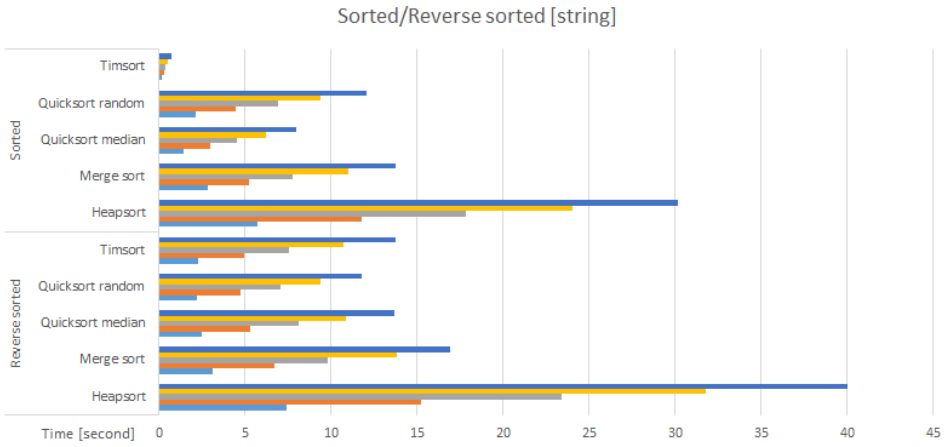
Figure 2.20: Average times on *sorted* and *reverse sorted* sequences of `uint_fast32_t`

with median of three pivot. For random sequences, the quicksort with last element pivot remained the fastest.

2.5.5 Conclusion points and final thoughts

- The *random* and *kmonotone* tests showed, that timsort is not necessarily faster than mergesort on inputs, that have a very low amount of presortedness. On nearly sorted and reverse sorted sequences, it performed only partially better. On all other sequence types, it was clearly superior to mergesort. Irrespective of minor differences on *random* sequences, timsort is a better choice.

- Timsort performs better on larger objects, that require more comparisons to figure out order of two elements. Due to its adaptiveness, it can compete with quicksorts on all sequences, that possess some level of presortedness.



	Reverse sorted					Sorted				
	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort	Heapsort	Merge sort	Quicksort median	Quicksort random	Timsort
50,000,000	40.05	16.90	13.72	11.79	13.78	30.16	13.77	7.97	12.07	0.71
40,000,000	31.76	13.84	10.89	9.41	10.75	24.02	11.03	6.23	9.41	0.55
30,000,000	23.38	9.85	8.14	7.08	7.55	17.85	7.75	4.57	6.93	0.41
20,000,000	15.26	6.69	5.34	4.77	4.93	11.80	5.26	3.00	4.45	0.28
10,000,000	7.39	3.12	2.48	2.23	2.31	5.76	2.88	1.45	2.17	0.14

Figure 2.21: Average times on *sorted* and *reverse sorted* sequences of `string`s, that are 32 letters long

- Quicksort with median pivot performed very well on all sequence types, despite being third on *reverse sorted* sequences of `uint_fast32_t`. Its only downside is, that it is vulnerable, because each time, it computes median from three same positioned elements. The random pivot is therefore a more secure choice, that performs similarly well on all inputs.

- Heapsort had the worst times of all algorithms measured in the third part. It was able to compete with mergesort only on *krestricted* sequences of `string`s, where it was slightly faster.

- Quicksort with last element pivot performs badly on all sequences, that have small amount of inversions. On nearly sorted or reverse sorted sequences, 8% of displaced elements were enough to prevent completely bad times. Compared to other quicksorts, it needed twice as much time on *nearly sorted* and up to four times on *nearly reverse sorted* sequences of $100,000,000$ elements. On

the *kmonotone* sequence type, it was just 15% slower. The only exception were *random* sequences, where it performed about one second better than quicksort with median of three pivot and about two seconds better than random pivot quicksort. The only factor, that could possibly play in favour of the last pivot strategy are its low costs.

- Binary insertion sort is faster than regular insertion sort on all sequence types, except for the *sorted* and *kshuffled* ones.
- Stooge sort is virtually unuseable. Even though it has lower than cubic complexity, its times were astronomically larger than those of insertion sort.

Measurings on `uint_fast32_t` type [second]						
Sequence type	Algorithms (with binary insertsort)	Sequence size				
		100,000,000	200,000,000	300,000,000	400,000,000	500,000,000
2monotone	Mergesort	13.23	27.48	42.34	57.07	72.33
	Quicksort median	10.72	22.23	34.01	45.90	57.80
	Quicksort random	10.82	22.38	34.03	46.05	59.11
4monotone	Mergesort	12.62	26.25	40.58	54.65	69.47
	Quicksort median	10.61	22.02	33.63	46.36	57.74
	Quicksort random	10.67	22.33	33.68	45.97	57.86
32monotone	Mergesort	11.32	24.16	36.66	49.47	62.72
	Quicksort median	10.03	21.13	32.03	43.25	54.71
	Quicksort random	10.16	22	31.90	43.39	54.95
16shuffled	Mergesort	4.53	9.33	14.53	19.20	24.07
	Quicksort median	3.67	7.46	11.38	15.24	19.11
	Quicksort random	4.18	8.67	12.95	17.37	21.83
32shuffled	Mergesort	4.93	10.11	15.82	21.36	25.86
	Quicksort median	3.94	8	12.17	17.47	20.25
	Quicksort random	4.44	9.09	13.86	19.64	23.13
64shuffled	Mergesort	5.36	10.97	16.88	22.40	28.16
	Quicksort median	4.17	8.49	12.85	17.25	21.56
	Quicksort random	4.71	9.57	14.73	19.58	24.41
Krestricted	Mergesort	6.99	14.38	22.46	29.71	37.29
	Quicksort median	3.95	8.06	12.45	16.70	20.91
	Quicksort random	4.08	8.25	12.57	17	21.30
Nearly reverse sorted	Mergesort	4.99	10.43	16.35	21.75	27.46
	Quicksort median	4.86	10.88	15.34	20.69	26.36
	Quicksort random	5.78	12	18.59	25.07	30.94
Nearly sorted	Mergesort	4.87	10.13	15.85	21.09	26.54
	Quicksort median	3	6.18	9.41	12.76	16.08
	Quicksort random	3.64	7.55	11.64	15.61	19.62
Random	Mergesort	13.78	28.11	44.41	59.04	74.64
	Quicksort last	10.51	21.39	33.22	44.95	56.56
	Quicksort median	10.86	22.31	34.38	46.52	58.49
	Quicksort random	10.86	22.35	34.82	46.22	58.55
Reverse sorted	Mergesort	4.12	8.65	15.18	19.84	22.77
	Quicksort median	2.90	6.07	9.92	12.64	15.88
	Quicksort random	2.70	5.60	8.74	11.58	14.51
Sorted	Mergesort	3.03	6.33	9.96	13.23	16.62
	Quicksort median	1.75	3.63	5.46	7.50	9.46
	Quicksort random	2.64	5.48	8.30	11.22	14.14

Figure 2.22: Average times for sorting algorithms, that were modified to use binary insertion sort on subsequences shorter or equal to 16

Measurings on `string` type [second]						
Sequence type	Algorithms (with binary insertsort)	Sequence size				
		10,000,000	20,000,000	30,000,000	40,000,000	50,000,000
2monotone	Mergesort	8.86	19.85	29.88	42.63	53.94
	Quicksort median	8.32	18.55	29.29	41.29	55.63
	Quicksort random	9.04	22.43	32.89	47.37	61.07
4monotone	Mergesort	11.92	15.06	23.65	35.43	49.83
	Quicksort median	7.06	15.21	25.34	34.29	42.38
	Quicksort random	7.36	16.67	26.70	38.96	45.61
32monotone	Mergesort	7.30	7.96	12.49	17.10	23.07
	Quicksort median	3.33	6.86	11	14.91	19.68
	Quicksort random	3.65	7.89	12.82	16.89	22.88
16shuffled	Mergesort	10.27	5.32	8.10	11.72	13.74
	Quicksort median	1.93	4.17	6.39	8.70	10.37
	Quicksort random	2.42	5.33	8.26	10.82	13.12
32shuffled	Mergesort	3.15	5.36	8.23	11.67	14.70
	Quicksort median	1.94	4.01	6.15	8.33	10.89
	Quicksort random	2.79	5.78	8.89	12.27	15.98
64shuffled	Mergesort	3.57	5.86	9.07	12.21	15.09
	Quicksort median	2.09	4.31	6.67	8.48	10.73
	Quicksort random	3.25	6.86	10.37	13.52	17.20
Krestricted	Mergesort	7.86	13.32	20.03	32.51	51.30
	Quicksort median	6.08	11.07	21.21	26.97	38.34
	Quicksort random	6.71	11.63	24.45	32.24	45.53
Nearly reverse sorted	Mergesort	4.77	8.60	14.60	20.74	28.31
	Quicksort median	2.70	5.90	10.31	13.60	18.87
	Quicksort random	3.90	8.79	16.75	21.12	27.98
Nearly sorted	Mergesort	6	8.73	13.07	20.38	25.61
	Quicksort median	2.27	4.95	7.76	11.86	15.24
	Quicksort random	3	6.45	9.94	14.67	18.71
Random	Mergesort	12.23	26.55	41.59	56.53	65.60
	Quicksort last	10.67	26.18	39.48	44.51	58.42
	Quicksort median	9.69	23.48	35.36	40.89	51.48
	Quicksort random	10.72	25.79	39.72	47.58	56.61
Reverse sorted	Mergesort	3.07	6.38	9.59	12.98	16.11
	Quicksort median	2.69	5.61	8.49	11.23	14.46
	Quicksort random	2.11	5.35	6.62	8.86	11.55
Sorted	Mergesort	2.66	4.74	7.07	10.44	12.85
	Quicksort median	1.46	3.02	4.75	6.61	8.44
	Quicksort random	1.95	4.09	6.37	8.75	11.47

Figure 2.23: Average times for sorting algorithms, that were modified to use binary insertion sort on subsequences shorter or equal to 16

3 Conclusion

The first part of the bachelor thesis is devoted to provide consistent overview of the chosen comparison-based sorting algorithms, ranging from the pessimal ones, up to those, that are currently being used in contemporary standard libraries. All of the chosen algorithms and their variants were described in detail in separate sections, that also include pseudocodes, discussion of stability and asymptotic space and time analysis in the worst, average and best case. However, due to missing relevant articles and publications of timsort, we have tried to approximate its overall costs in the worst and average case by summing costs of the merging and presorting phase. Furthermore, the selected algorithm variants, that perform optimally were implemented in C++.

The second part of the thesis is dedicated to the detailed performance analysis. All of the implemented algorithm variants have undergone several performance analysis tests, that involved 8 types of sequences with a different kind and level of presortedness. Some of the sequence types were even parametrized by additional variable, that depending on a sequence type specified the level of presortedness. The results obtained by the provided *benchmark* program were processed and discussed at the end of the work.

The measurements showed multiple interesting properties. From among them, for example, that timsort is not necessarly faster than the bottom-up mergesort. The main reason was, that timsort's adaptive features didn't pay off on sequences, that contained a very low amount of presortedness. In addition, the results revealed, that it is competetive with quicksorts for larger objects, that are costly to compare. The best overall performer for all sequence and element types was quicksort with median of three pivot. On the contrary, the worst performing one was bogosort.

Bibliography

1. BRODER, Andrei; STOLFI, Jorge. Pessimal Algorithms and Simplexity Analysis. 1986.
2. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. In: 3rd. Cambridge, MA, U.S.A: MIT Press, 2009, pp. 192–193.
3. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. In: 2nd. Cambridge, MA, U.S.A: MIT Press, 2001, pp. 161–162.
4. WILLIAMS, J. W. J. Algorithm 232: Heapsort. *Communications of the ACM*. 1964, vol. 7, no. 6, pp. 347–348.
5. FLOYD, Robert W. Algorithm 245: Treesort. *Commun. ACM*. 1964, vol. 7, no. 12, pp. 701–. ISSN 0001-0782.
6. CARLSSON, Svante. A Variant of Heapsort with Almost Optimal Number of Comparisons. *Inf. Process. Lett.* 1987, vol. 24, no. 4, pp. 247–250. ISSN 0020-0190.
7. WEGENER, Ingo. BOTTOM-UP-HEAPSORT, a new variant of HEAPSORT beating, on an average, QUICKSORT (if n is not very small). *Theoretical Computer Science*. 1993, vol. 118, no. 1, pp. 81–98. ISSN 0304-3975.
8. CARLSSON, Svante. Average-case results on heapsort. *BIT Numerical Mathematics*. 1987, vol. 27, no. 1, pp. 2–17.
9. HOARE, C. A. R. Algorithm 64: Quicksort. *Commun. ACM*. 1961, vol. 4, no. 7, pp. 321–. ISSN 0001-0782.
10. HOARE, C. A. R. Quicksort. *The Computer Journal*. 1962, vol. 5, no. 1, pp. 10–16.
11. SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms*. 4th. Addison-Wesley Professional, 2011. ISBN 032157351X.
12. SINGLETON, Richard C. Algorithm 347: An Efficient Algorithm for Sorting with Minimal Storage [M1]. *Commun. ACM*. 1969, vol. 12, no. 3, pp. 185–186. ISSN 0001-0782.

13. BENTLEY, Jon L.; MCILROY, M. Douglas. Engineering a Sort Function. *Softw. Pract. Exper.* 1993, vol. 23, no. 11, pp. 1249–1265. ISSN 0038-0644.

14. SEDGEWICK, Robert. Implementing Quicksort Programs. *Commun. ACM*. 1978, vol. 21, no. 10, pp. 847–857. ISSN 0001-0782.

15. KNUTH, Donald E. The Art of Computer Programming, Volume 2 (3rd Ed.): Seminumerical Algorithms. In: Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1997, pp. 145–146. ISBN 0-201-89684-2.

16. STEIN, Prof. Cliff. *Analysis of Algorithms I Lecture*. University of Columbia, 116th St & Broadway, New York, NY 10027, USA, 2009. Available also from: `http://www.columbia.edu/~cs2035/courses/csor4231.F09/rand.pdf`. Accessed: 2017-10-09.

17. GRUBER, Hermann; HOLZER, Markus; RUEPP, Oliver. Sorting the Slow Way: An Analysis of Perversely Awful Randomized Sorting Algorithms. In: *Fun with Algorithms: 4th International Conference, FUN 2007, Castiglioncello, Italy, June 3-5, 2007. Proceedings*. Ed. by CRESCENZI, Pierluigi; PRENCIPE, Giuseppe; PUCCI, Geppino. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 183–197. ISBN 978-3-540-72914-3.

18. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 3rd. Cambridge, MA, U.S.A: MIT Press, 2009.

19. GOLDSTINE, H.H.; VON NEUMANN, J. Planning and Coding of Problems for an Electronic Computing Instrument: Report on the mathematical and logical aspects of an electronic computing instrument. In: Institute for Advanced Study, 1948, vol. 2, pp. 152–214. No. II.

20. KNUTH, Donald E. *The Art of Computer Programming, Volume 3: (2Nd Ed.) Sorting and Searching*. Redwood City, CA, USA: Addison Wesley Longman Publishing Co., Inc., 1998. ISBN 0-201-89685-0.

21. KATAJAINEN, Jyrki; PASANEN, Tomi; TEUHOLA, Jukka. Practical In-place Mergesort. *Nordic J. of Computing*. 1996, vol. 3, no. 1, pp. 27–40. ISSN 1236-6064.

22. HUANG, Bing-Chao; LANGSTON, Michael A. Practical In-place Merging. *Commun. ACM*. 1988, vol. 31, no. 3, pp. 348–352. ISSN 0001-0782.

23. REINHARDT, Klaus. Sorting in-place with a worst case complexity of n log n–1.3n+O(log n) comparisons and ϵ n log n+O(1) transports. In: *Algorithms and Computation: Third International Symposium, ISAAC'92 Nagoya, Japan, December 16–18, 1992 Proceedings*. Ed. by IBARAKI, Toshihide; INAGAKI, Yasuyoshi; IWAMA, Kazuo; NISHIZEKI, Takao; YAMASHITA, Masafumi. Berlin, Heidelberg: Springer Berlin Heidelberg, 1992, pp. 489–498. ISBN 978-3-540-47501-9.

24. FLAJOLET, P.; GOLIN, M. The mergesort recurrence. In: *Mellin transforms and asymptotic*. 1994, pp. 673–696.

25. PANNY, W.; PRODINGER, H. Bottom-up mergesort — A detailed analysis. *Algorithmica*. 1995, vol. 14, no. 4, pp. 340–354. ISSN 1432-0541.

26. PETERS, Tim. *Timsort description*. Available also from: `https://svn.python.org/projects/python/trunk/Objects/listsort.txt`. Accessed: 2017-09-25.

27. MCILROY, Peter. Optimistic Sorting and Information Theoretic Complexity. In: *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*. Austin, Texas, USA: Society for Industrial and Applied Mathematics, 1993, pp. 467–474. SODA '93. ISBN 0-89871-313-7.

28. PETERSSON, Ola; MOFFAT, Alistair. A framework for adaptive sorting. *Discrete Applied Mathematics*. 1995, vol. 59, no. 2, pp. 153–179. ISSN 0166-218X.

29. MANNILA, Heikki. Measures of Presortedness and Optimal Sorting Algorithms. *IEEE Trans. Comput.* 1985, vol. 34, no. 4, pp. 318–325. ISSN 0018-9340.

30. AUGER, Nicolas; NICAUD, Cyril; PIVOTEAU, Carine. *Merge Strategies: from Merge Sort to TimSort*. 2015. Available also from: `https://hal-upec-upem.archives-ouvertes.fr/hal-01212839`.

31. KROAH-HARTMAN, Greg. Linux Kernel in a Nutshell. In: O'Reilly Media, Inc., 2006, p. 97. ISBN 0596100795.

32. LOVE, Robert. Linux System Programming: Talking Directly to the Kernel and C Library. In: O'Reilly Media, Inc., 2007, pp. 187–189. ISBN 0596009585.

33. BOVET, Daniel; CESATI, Marco. *Understanding The Linux Kernel*. Oreilly & Associates Inc, 2005. ISBN 0596005652.

34. *Date and time utilities*. Available also from: `http://en.cppreference.com/w/cpp/chrono`. Accessed: 2017-23-10.

Appendices

File attachments:

The zip file called *Source_codes.zip* contains following:

```
Source_codes/
├── benchmark.h
├── binary_insertsort.h
├── bogosort.h
├── CMakeLists.txt
├── heapsort.h
├── insertsort.h
├── iterator_alias.h
├── main.cpp
├── mergesort.h
├── quicksort.h
├── stoogesort.h
└── timsort.h
```